DATA 260: Agentic AI

Instructor: Prof. Dr. Simon Shim

Student Name: Trapti Kulshrestha

STUDENT ID: 019124833

Homework: 4

Section 0: Personal Configuration:

SID4	4833
PORT_BASE:	8333
PREFIX	s4833
SEED	4833
VERIFY_SEED	264833
DOMAIN_ID:	1
Assigned Domain	Clinical Trial Listings

Hardware: Apple MacBook Air M4

Local Model: qwen3:8b

Tagged Commit Hash:

GitHub Repository: https://github.com/Psycoder0611/data260-4833

Project Setup:

For Homework 4, I continued working on my existing DATA 260 repository rather than creating a separate project. I created a new working branch named hw4-work to keep my previous homework work separate from the new implementation.

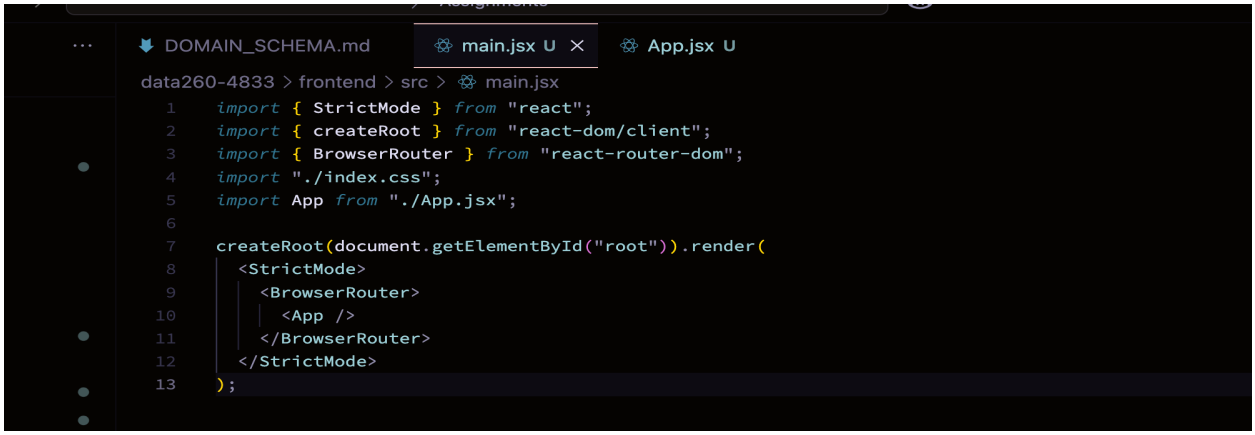
I also created the reports/hw04/ directory and its raw/ subdirectory to organize my report, experiment results, and supporting evidence.

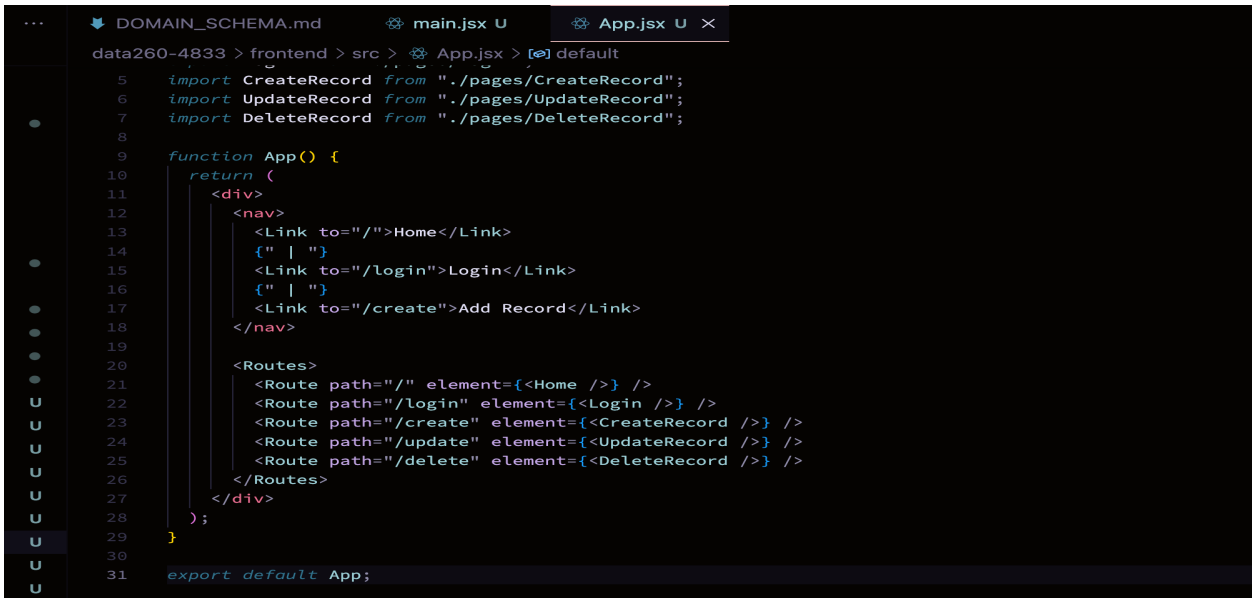
Part 1 – React Client

1.1 React Client Setup and Routing

I created the React client using Vite inside the existing repository and verified that the development server was running successfully on localhost:5173.

I wrapped the application with BrowserRouter so React Router can manage navigation between the required pages.





Result / Observation:

The React application successfully navigates between the Home, Login, Create, Update, and Delete routes using React Router.

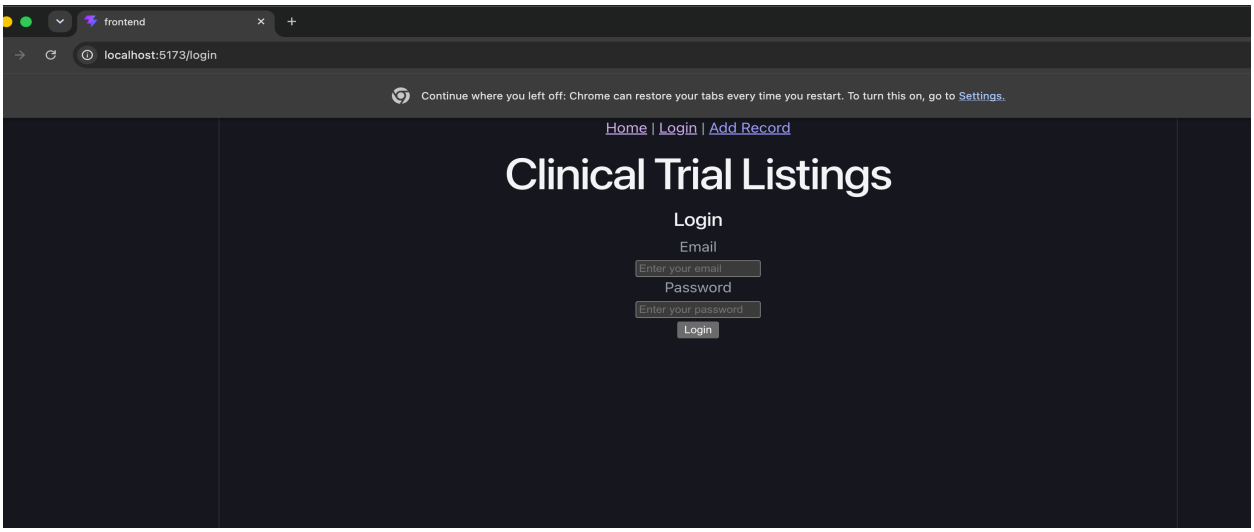
1.2 Login Component – Login.jsx

I created the Login.jsx component with email and password fields and connected it to the FastAPI login endpoint. I used React useState to store the values entered by the user and Axios to send the login request to /api/login.

The request uses withCredentials: true so the browser can receive and send the HTTP-only session cookie created by the backend. If the login is successful, the user is redirected to the Home page. If the credentials are incorrect, an error message is shown on the login page.







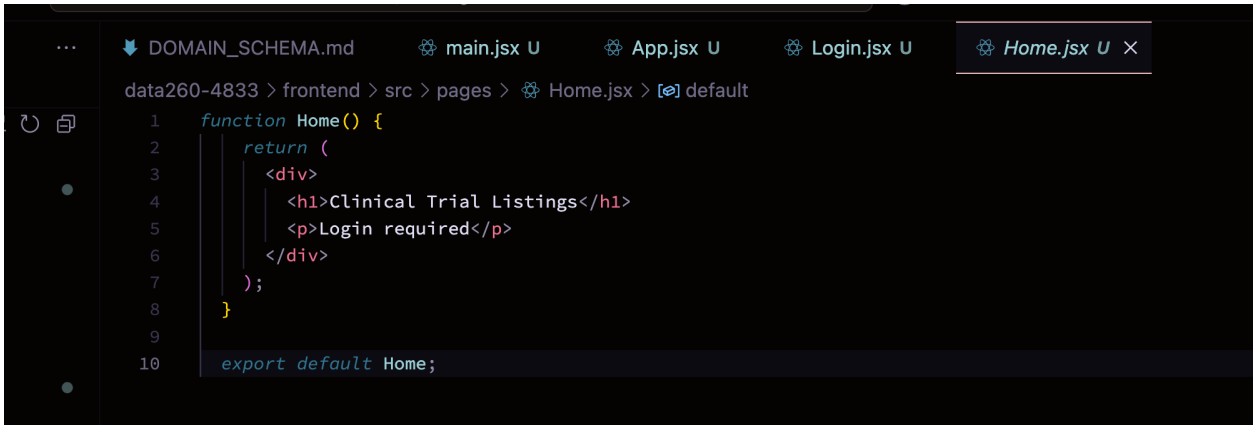
Screenshot : localhost:5173/login showing the complete login form.

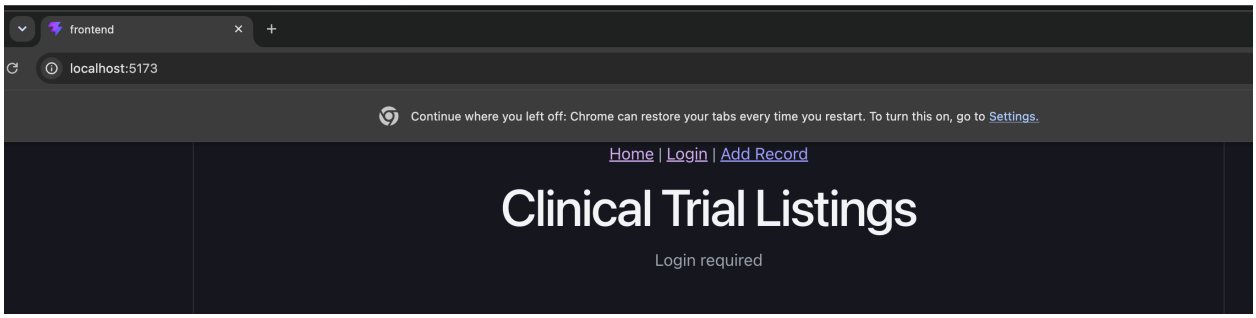
Result / Observation: The Login component successfully authenticates the user through FastAPI. After a successful login, the browser receives the HTTP-only session cookie and the user is redirected to the protected Home page.

1.3 Home Page – Home.jsx

1.3A Unauthenticated Home

The Home.jsx component is rendered at the root route /. At this stage, when there is no authenticated session, the page displays the message Login required. This ensures that protected clinical trial data is not shown to an unauthenticated user.





Result / Observation:

The Home page correctly displays Login required when the application is accessed without an authenticated session.

1.3B Authenticated Home / Record List:

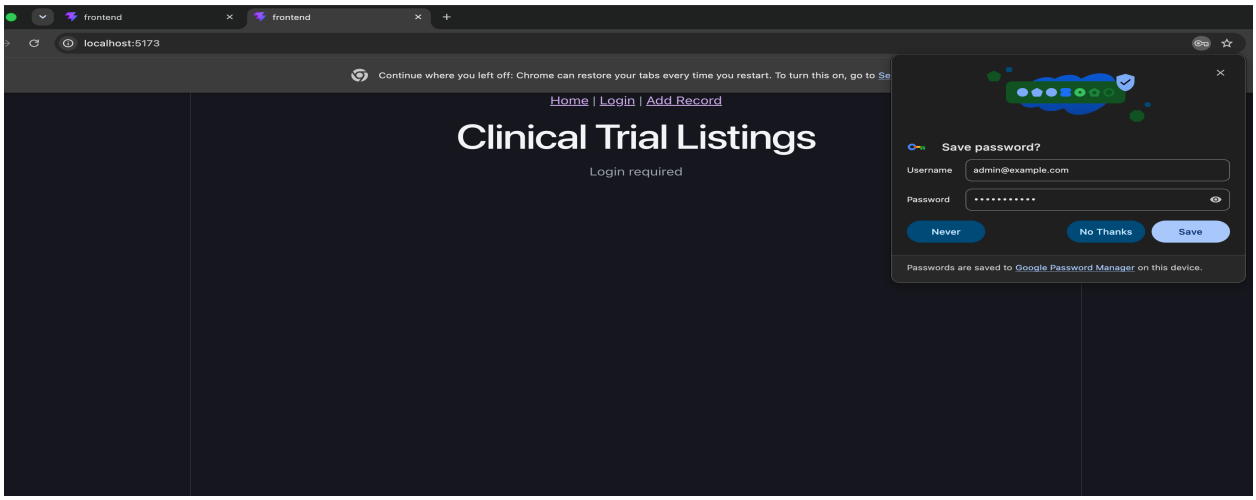
After a successful login, the Home component verifies the authenticated session through the FastAPI /api/me endpoint and then requests the clinical trial records from /api/hw4/trials. Because the test records had already been deleted during CRUD testing, the authenticated Home page correctly displayed No clinical trials found.

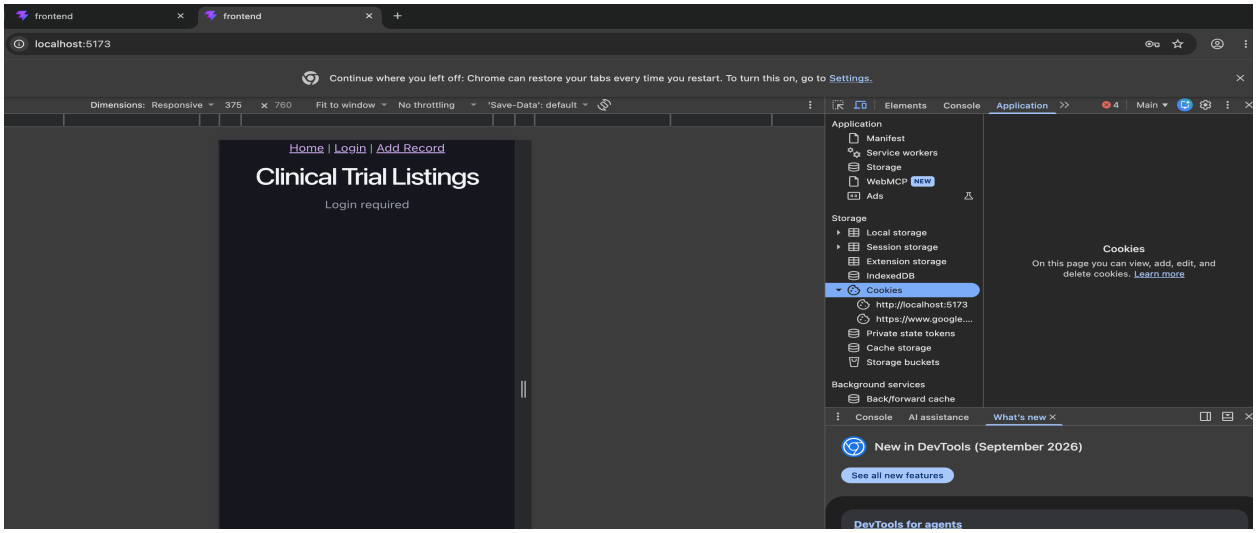
```
    try {
      await axios.get(
        "http://localhost:8333/api/me",
        {
          withCredentials: true,
        }
      );

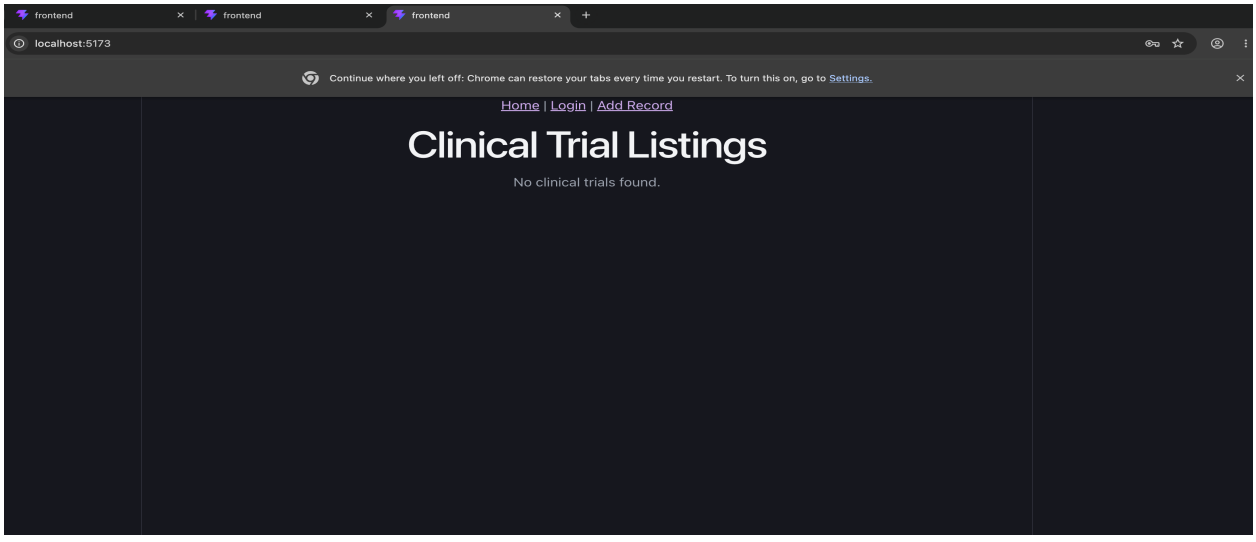
      setIsAuthenticated(true);

      const response = await axios.get(
        "http://localhost:8333/api/hw4/trials",
        {
          withCredentials: true,
        }
      );

      setRecords(response.data);
    } catch (error) {
      setIsAuthenticated(false);
    }
  };
```







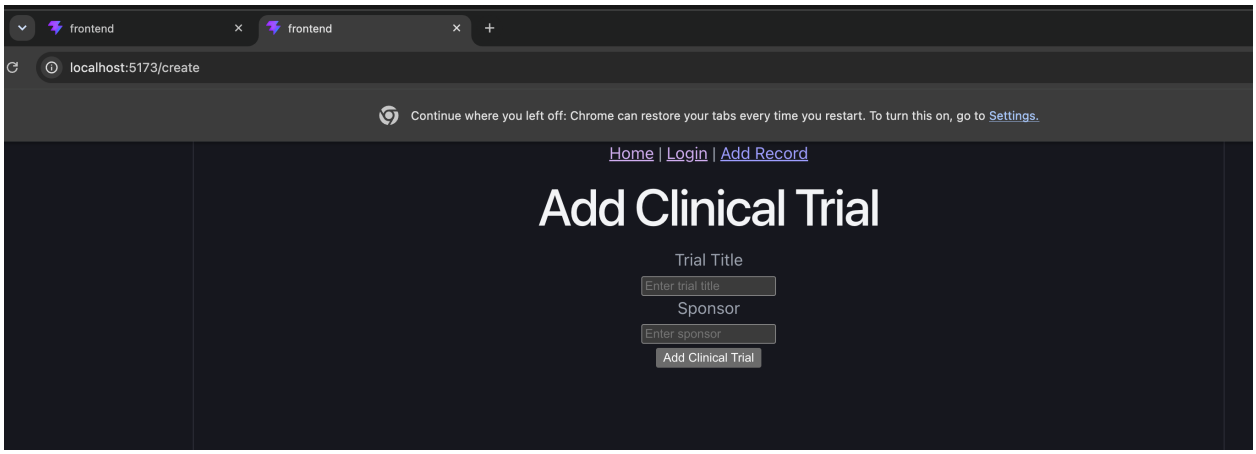
Screenshot: Authenticated Home page successfully connected to FastAPI/MySQL with no current clinical trial records.

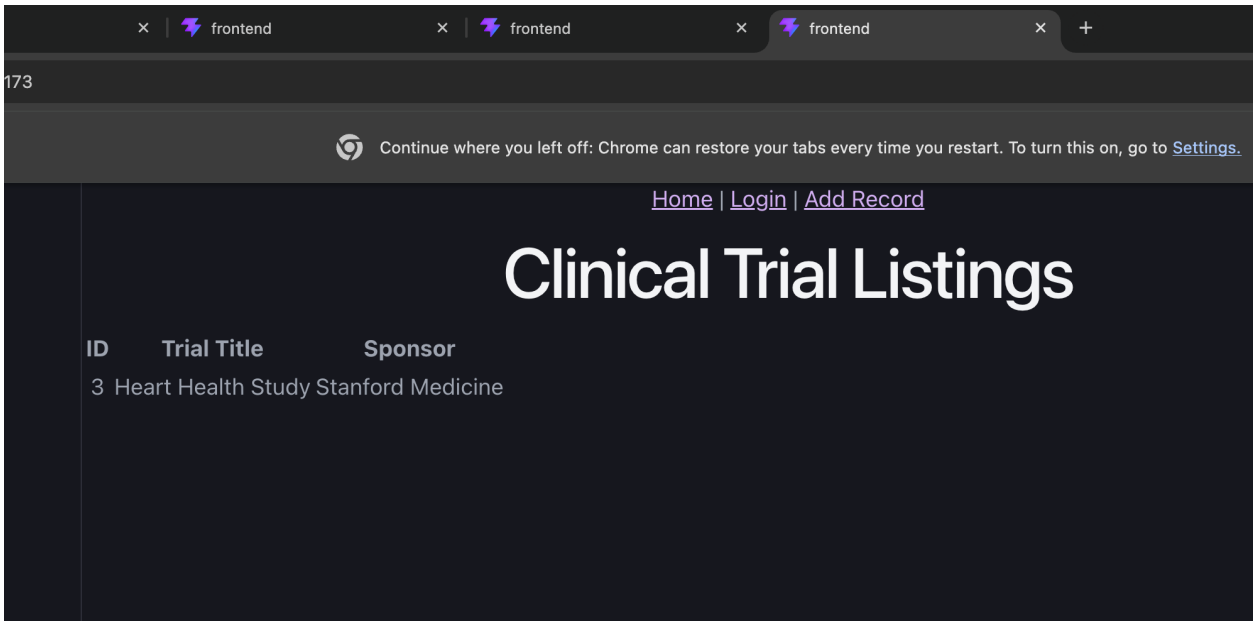
1.4 Create Record – CreateRecord.jsx

I connected CreateRecord.jsx to the protected FastAPI POST endpoint using Axios. The component sends the trial title and sponsor to /api/hw4/trials with withCredentials: true, allowing the browser to include the HTTP-only session cookie. After successful creation, the user is redirected to the Home page.









Screenshot : Home page showing the new record after submission

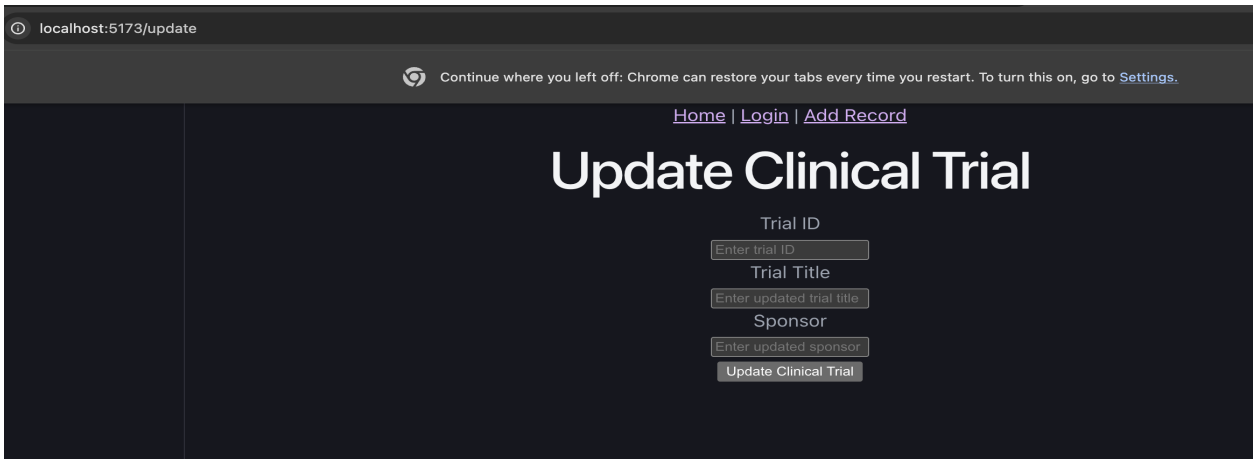
Result / Observation: The Create Record form successfully sends the clinical trial title and sponsor to the protected FastAPI POST endpoint. The record is persisted in MySQL, and after successful creation the user is redirected to the Home page where the new record is displayed.

1.5 Update Record:

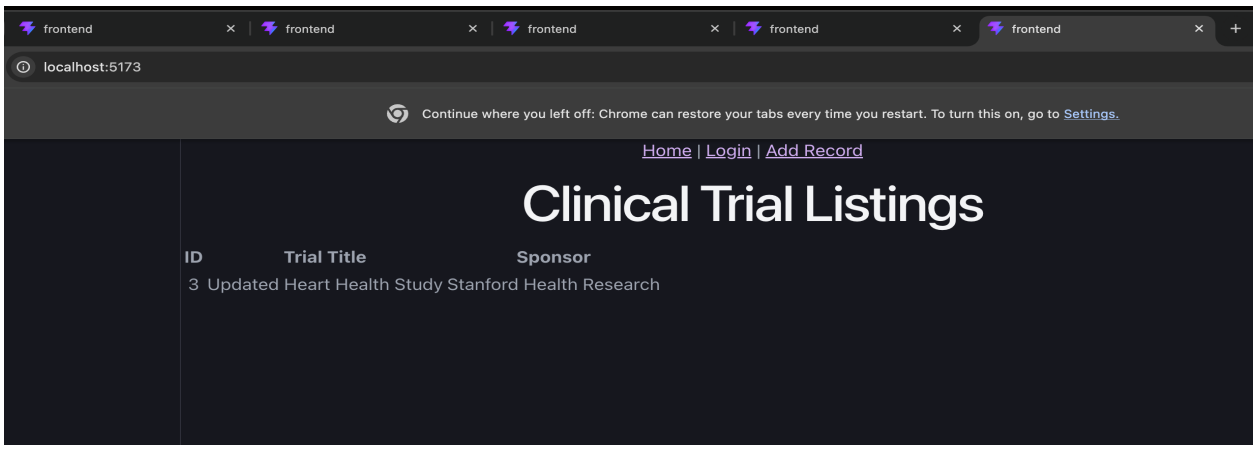
I connected UpdateRecord.jsx to the protected FastAPI PUT endpoint. The component sends the selected trial ID together with the updated trialTitle and sponsor values. After the backend commits the change to MySQL, the user is redirected to the Home page.

```
    try {
      await axios.put(
        `http://localhost:8333/api/hw4/trials/${id}`,
        {
          trialTitle: trialTitle.trim(),
          sponsor: sponsor.trim(),
        },
        {
          withCredentials: true,
        }
      );

      navigate("/");
    } catch (err) {
      if (err.response?.status === 401) {
        setError("Login required");
      } else if (err.response?.status === 404) {
        setError("Clinical trial not found");
      } else {
        setError("Unable to update clinical trial");
      }
    }
```



Screenshot: Update form filled in before submission



Screenshot: Home page showing the updated record

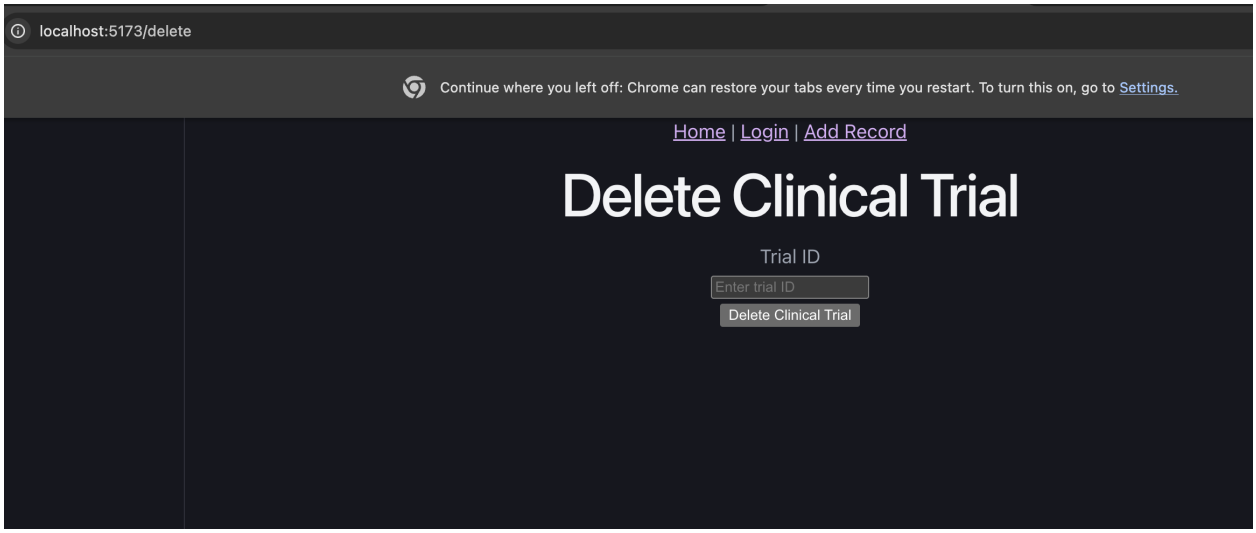
Result / Observation: The selected clinical trial was successfully updated through the FastAPI backend and the modified values were displayed on the Home page.

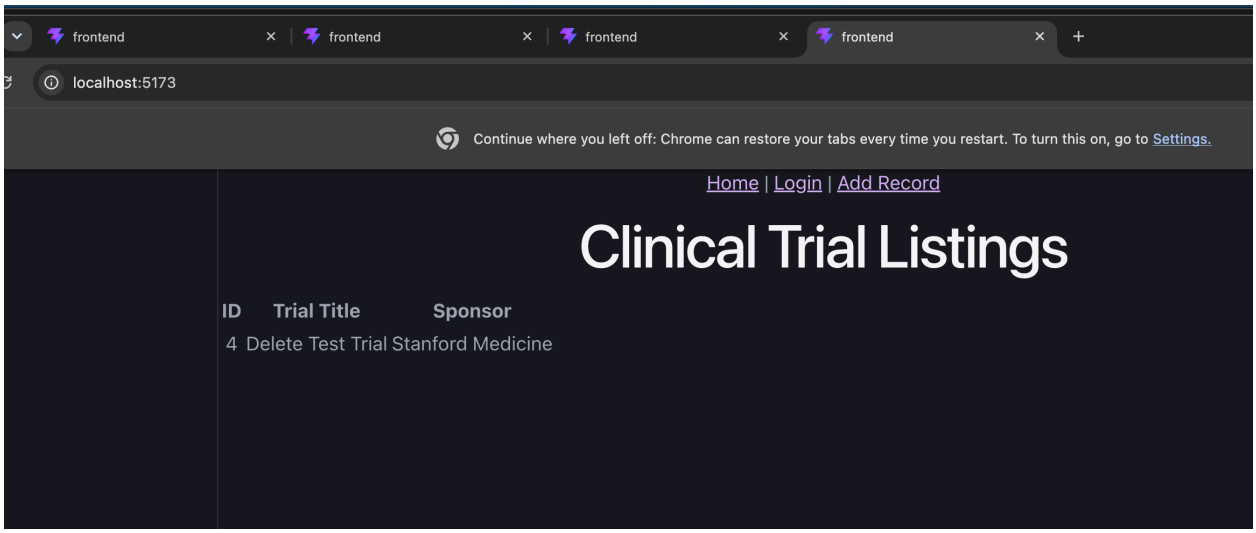
1.6 Delete Record:

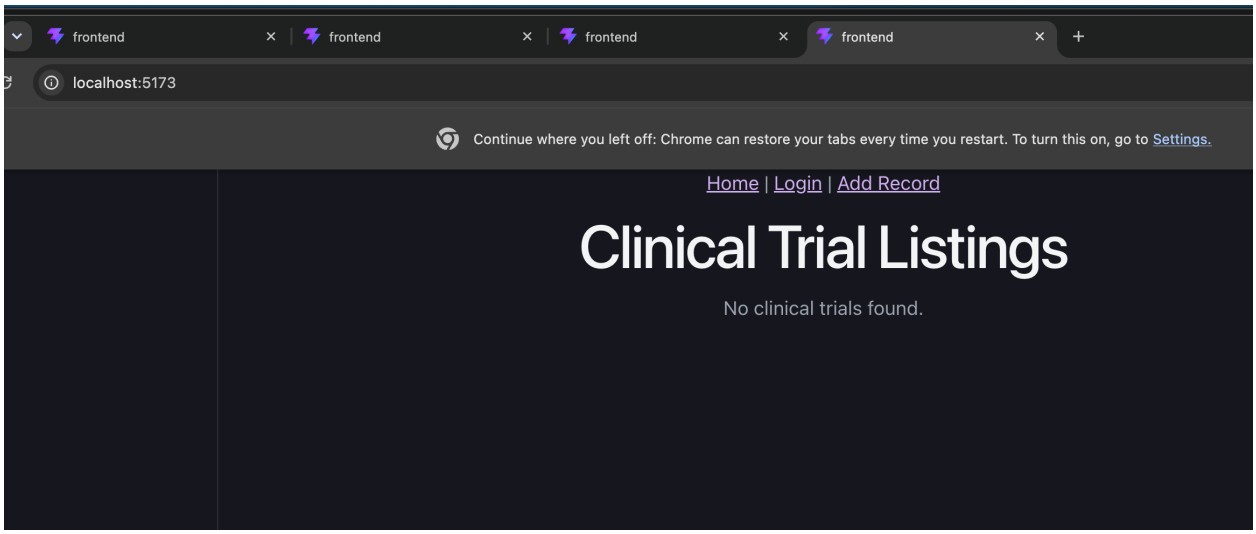
I connected DeleteRecord.jsx to the protected FastAPI DELETE endpoint. The component accepts the selected clinical trial ID and sends an authenticated DELETE request to the backend. After the record is removed from MySQL, the user is redirected to the Home page.



```
14
15    try {
16      await axios.delete(
17        `http://localhost:8333/api/hw4/trials/${id}`,
18        {
19          withCredentials: true,
20        }
21      );
22
23      navigate("/");
24    } catch (err) {
25      if (err.response?.status === 401) {
26        setError("Login required");
27      } else if (err.response?.status === 404) {
28        setError("Clinical trial not found");
29      } else {
30        setError("Unable to delete clinical trial");
31      }
32    }
33  };
34
35  return (
```







Screenshot: Home page after successful deletion, confirming that the clinical trial record is no longer present.

Result / Observation: The selected clinical trial was successfully deleted through the FastAPI backend. After deletion, the user was redirected to the Home page, where the record was no longer displayed.

1.7 React Hooks, Props, and Router:

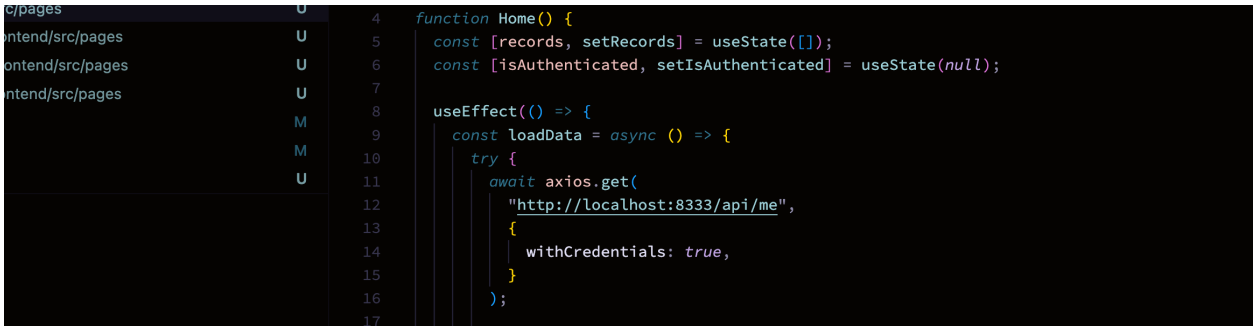
I used React hooks and React Router throughout the client application. useState manages form inputs, authentication errors, and record data, while useEffect loads authenticated clinical trial data when the Home component is rendered. useNavigate redirects the user after successful login and CRUD operations.

I also passed the shared apiBaseUrl value from App.jsx to the Create, Update, and Delete components through props. React Router defines separate routes for Home, Login, Create, Update, and Delete pages.

Props:



Hooks:



Result / Observation: React hooks successfully manage component state, data loading, and navigation. Props are passed from the parent App component to the Create, Update, and Delete components, and React Router provides navigation between all required pages.

PART-2:

MySQL Database Setup:

For this assignment, I used MySQL 9.4.0, installed through Homebrew on my MacBook.

I started by checking whether MySQL was installed and whether the database server was running. After confirming that the service was active, I connected to MySQL using the root account.

I created a new database named s4833_rel using my assigned prefix, as specified in the homework instructions.

I then selected the database and verified the active database name and MySQL server version.

SQL commands:

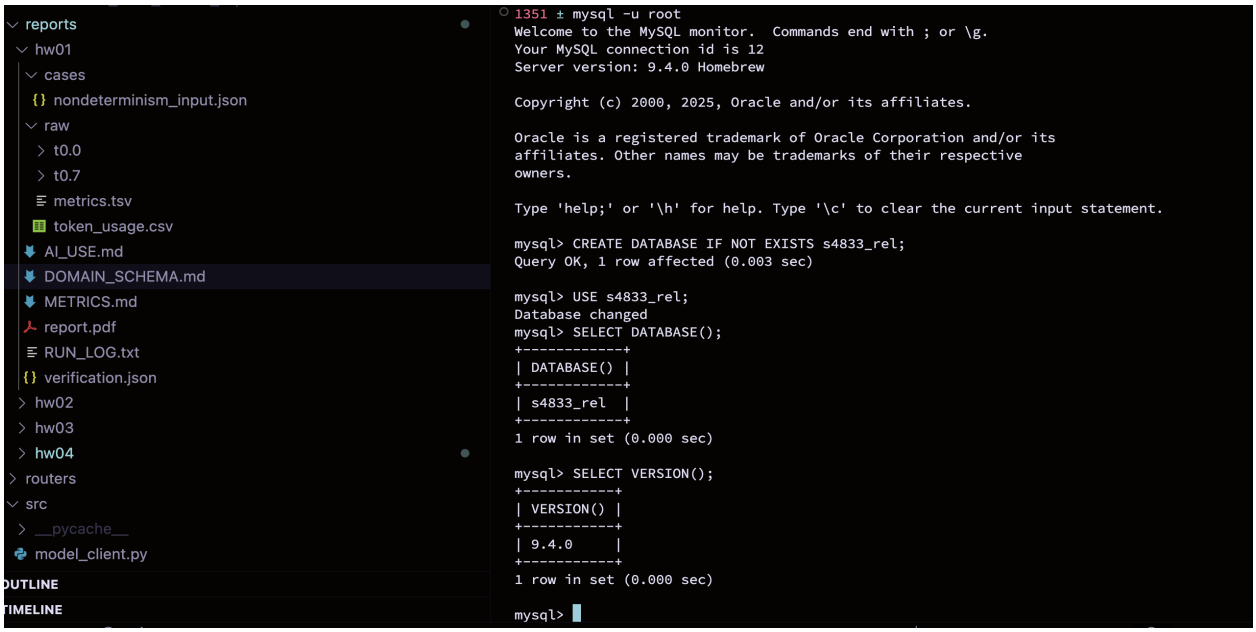
CREATE DATABASE IF NOT EXISTS s4833_rel;

USE s4833_rel;

SELECT DATABASE();

SELECT VERSION();

Output: The database was created successfully. The database verification returned s4833_rel, and the server version was 9.4.0.



Screenshot 1: MySQL database creation and verification.

2.2 SQLAlchemy Database Connection:

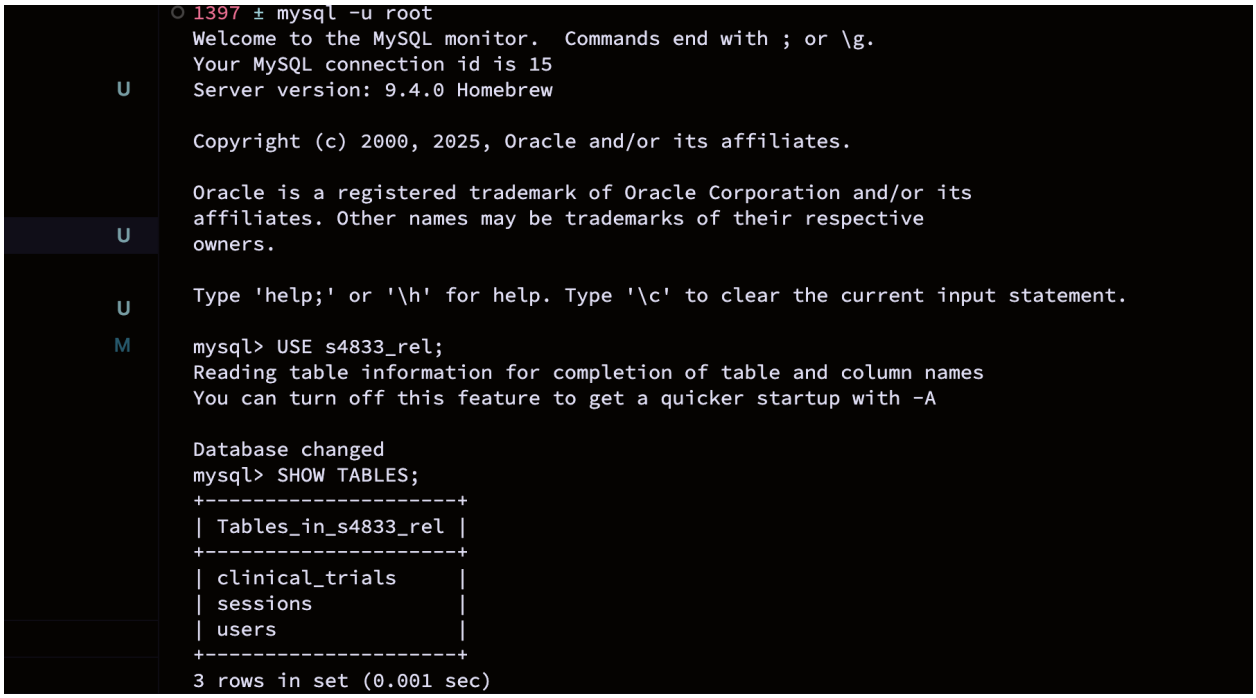
I connected the FastAPI application to the s4833_rel MySQL database using SQLAlchemy and PyMySQL. I created the required db_session_basede26 session factory so API requests can open and close database sessions safely.



Screenshot: SQLAlchemy connection successfully verified against the s4833_rel database.

2.3 Primary Domain Table:

I created the clinical_trials table using a SQLAlchemy model. The id field is configured as an auto-incrementing primary key. The primary domain field is trialTitle and the secondary domain field is sponsor. I retained the additional clinical trial fields from my previous homework so the existing domain application can continue to be extended.



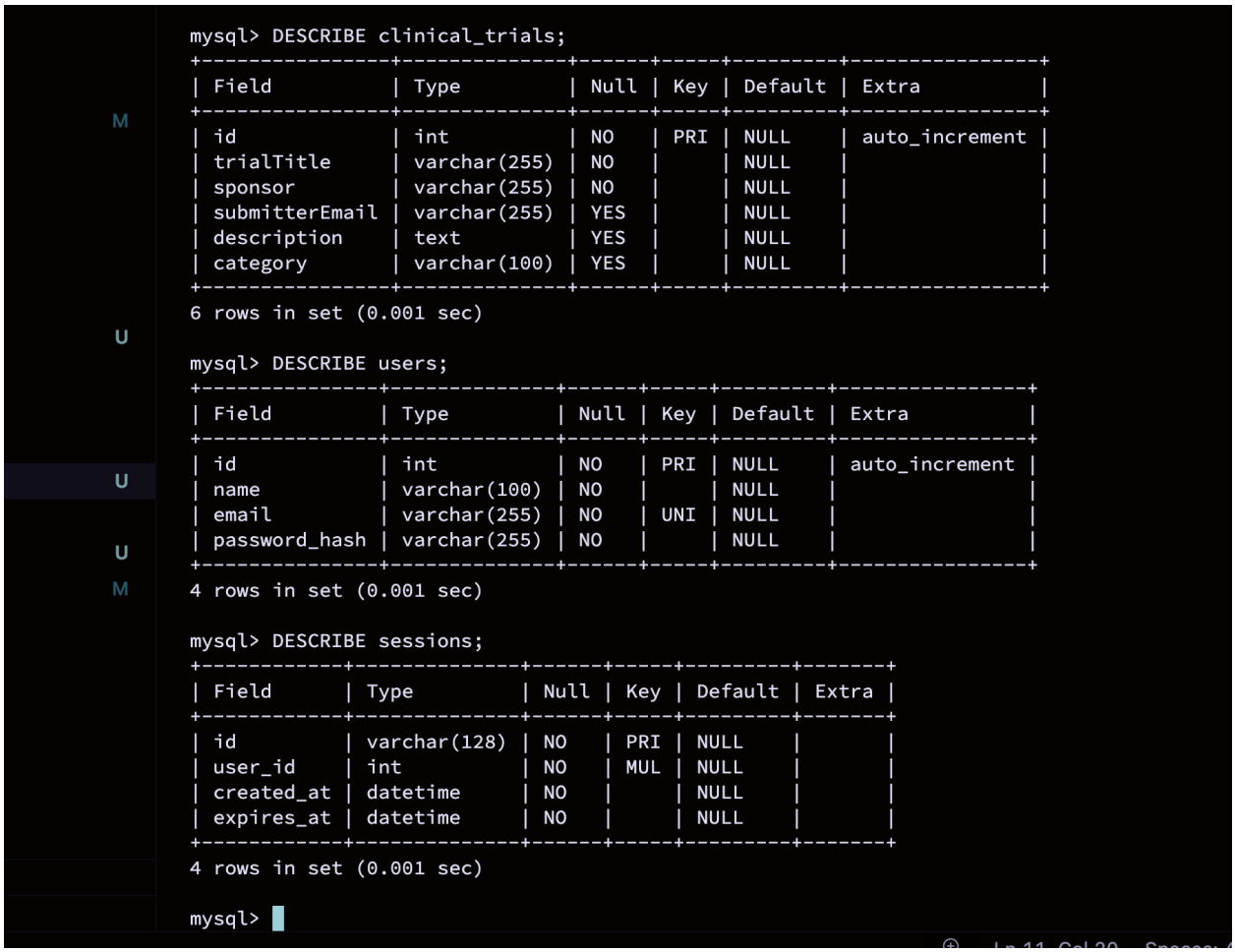
Screenshot: Tables created

2.4 Users Table

I created the users table for authentication. It contains an auto-incrementing id, the user's name, a unique email, and a password_hash. The table is designed so that passwords are stored as hashes rather than plain text.

2.5 Sessions Table

I created the sessions table for server-side session management. The id field stores the opaque session token, user_id links the session to the authenticated user, and created_at and expires_at store the session lifetime information.



```
mysql> DESCRIBE clinical_trials;
+----------------+--------------+------+-----+---------+----------------+
| Field          | Type         | Null | Key | Default | Extra          |
+----------------+--------------+------+-----+---------+----------------+
| id             | int          | NO   | PRI | NULL    | auto_increment |
| trialTitle     | varchar(255) | NO   |     | NULL    |                |
| sponsor        | varchar(255) | NO   |     | NULL    |                |
| submitterEmail | varchar(255) | YES  |     | NULL    |                |
| description    | text         | YES  |     | NULL    |                |
| category       | varchar(100) | YES  |     | NULL    |                |
+----------------+--------------+------+-----+---------+----------------+
6 rows in set (0.001 sec)

mysql> DESCRIBE users;
+---------------+--------------+------+-----+---------+----------------+
| Field         | Type         | Null | Key | Default | Extra          |
+---------------+--------------+------+-----+---------+----------------+
| id            | int          | NO   | PRI | NULL    | auto_increment |
| name          | varchar(100) | NO   |     | NULL    |                |
| email         | varchar(255) | NO   | UNI | NULL    |                |
| password_hash | varchar(255) | NO   |     | NULL    |                |
+---------------+--------------+------+-----+---------+----------------+
4 rows in set (0.001 sec)

mysql> DESCRIBE sessions;
+------------+--------------+------+-----+---------+-------+
| Field      | Type         | Null | Key | Default | Extra |
+------------+--------------+------+-----+---------+-------+
| id         | varchar(128) | NO   | PRI | NULL    |       |
| user_id    | int          | NO   | MUL | NULL    |       |
| created_at | datetime     | NO   |     | NULL    |       |
| expires_at | datetime     | NO   |     | NULL    |       |
+------------+--------------+------+-----+---------+-------+
4 rows in set (0.001 sec)

mysql>
```

Screenshot: MySQL table structures for clinical_trials, users, and sessions.

Result / Observation: The required MySQL tables were created successfully. The clinical_trials table contains the domain fields with an auto-incrementing primary key, the users table contains the required authentication fields, and the sessions table contains the fields needed for server-side session tracking.

Only the opaque session token will be sent to the browser in an HTTP-only cookie, while the user and session data remain stored server-side in MySQL.

2.6 Login + HTTP-only Server-Side Session:

I created a test user in the users table and stored the password as a hash rather than plain text. This user will be used to verify the server-side login and session flow.

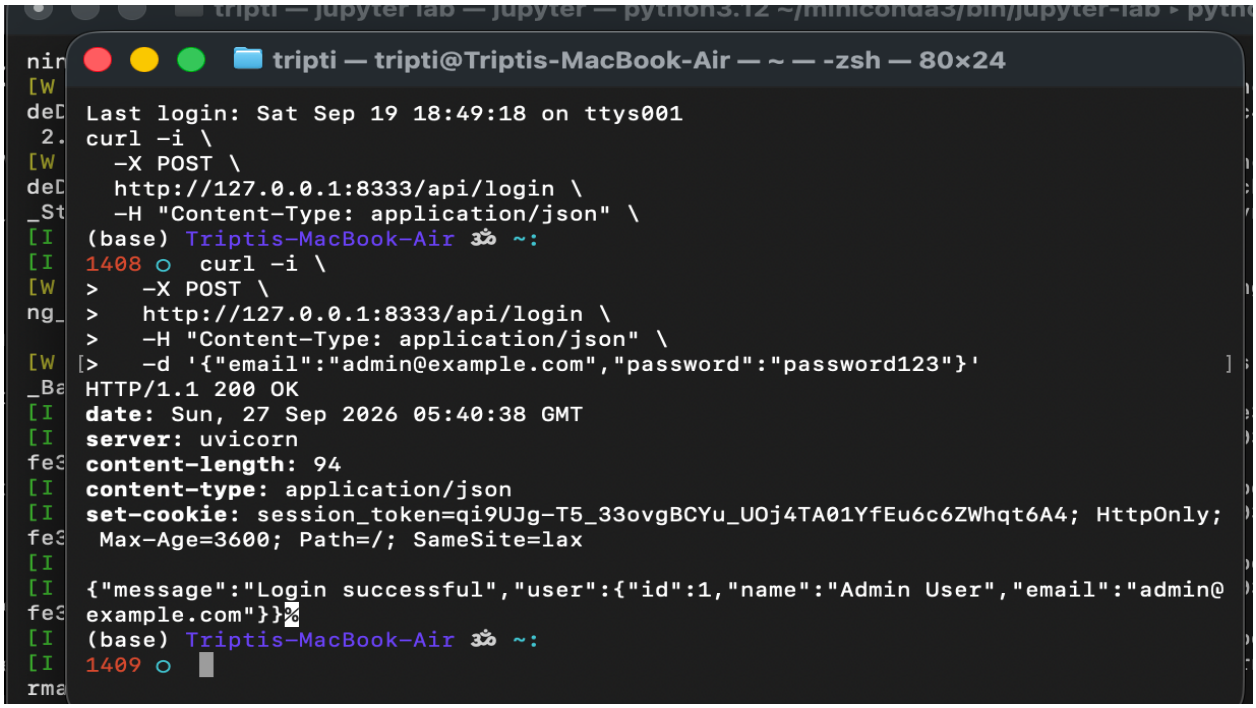
I implemented the HW4 login endpoint at /api/login. The backend looks up the user by email and verifies the entered password against the stored password hash. After successful

authentication, the server generates a random opaque session token and stores the session in the MySQL sessions table with its creation and expiration times.

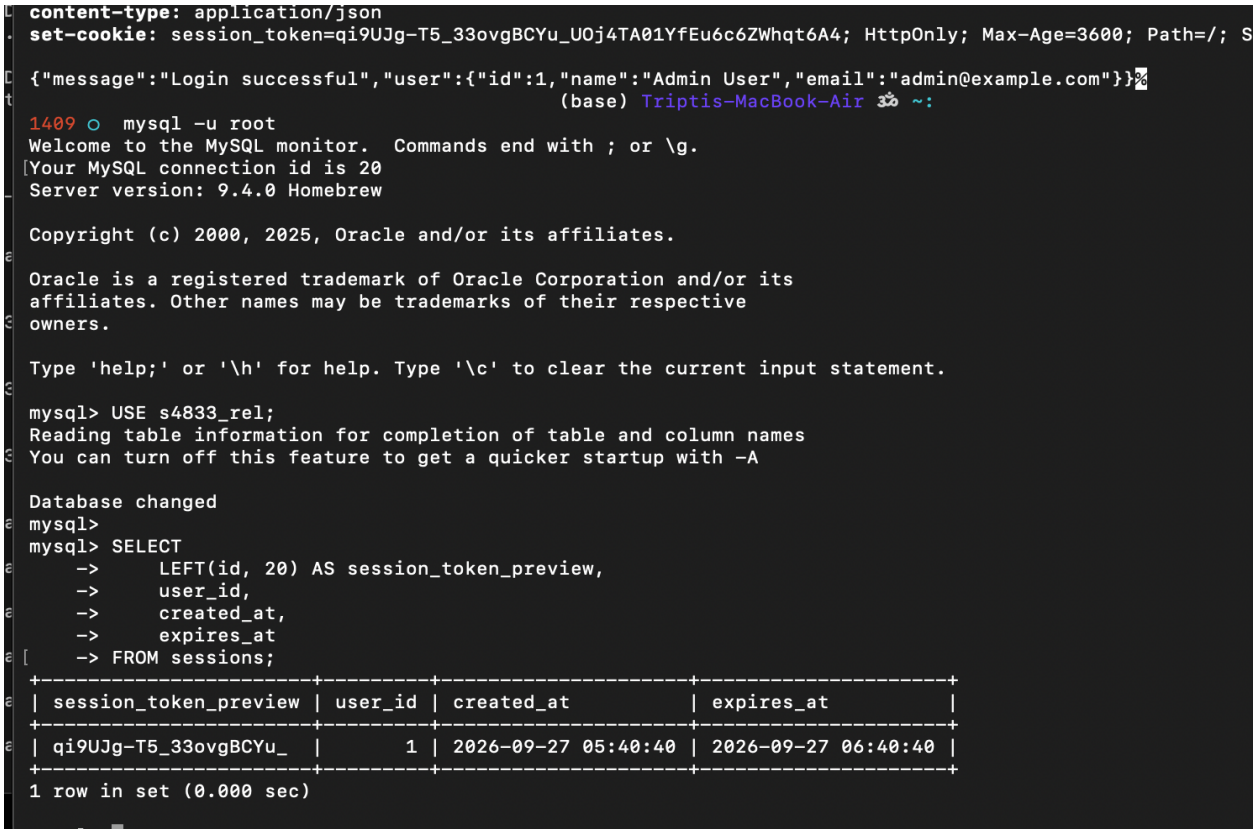
The same opaque token is returned to the browser as an HTTP-only cookie. The cookie does not contain the user's name, email, password, or other user data. The actual session state remains stored server-side in MySQL.



Screenshot: Test user stored in the users table with a hashed password.



Screenshot : Successful login response with HTTP-only session cookie.



Screenshot: Server-side session stored in the MySQL sessions table.

Browser cookie: opaque token only

MySQL: actual server-side session information

```python
        )

    session_token = secrets.token_urlsafe(32)

    created_at = datetime.utcnow()
    expires_at = created_at + timedelta(hours=1)

    session = SessionDB(
        id=session_token,
        user_id=user.id,
        created_at=created_at,
        expires_at=expires_at
    )

    db.add(session)
    db.commit()

    response.set_cookie(
        key="session_token",
        value=session_token,
        httponly=True,
        samesite="lax",
        secure=False,
        max_age=3600
    )
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

[notice] A new release of pip is available: 24.3.1 -> 26.2.1
[notice] To update, run: pip install --upgrade pip
(.venv) (base) Triptis-MacBook-Air ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(
1407 ± uvicorn main:app --host 127.0.0.1 --port 8333 --reload
INFO:     Will watch for changes in these directories: ['/Users/tripti/SJSU_Study/DATA_260_Agenti
]
INFO:     Uvicorn running on http://127.0.0.1:8333 (Press CTRL+C to quit)
INFO:     Started reloader process [67592] using StatReload
INFO:     Started server process [67594]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:62245 - "POST /api/login HTTP/1.1" 200 OK
```

```python
    response.set_cookie(
        key="session_token",
        value=session_token,
        httponly=True,
        samesite="lax",
        secure=False,
        max_age=3600
    )

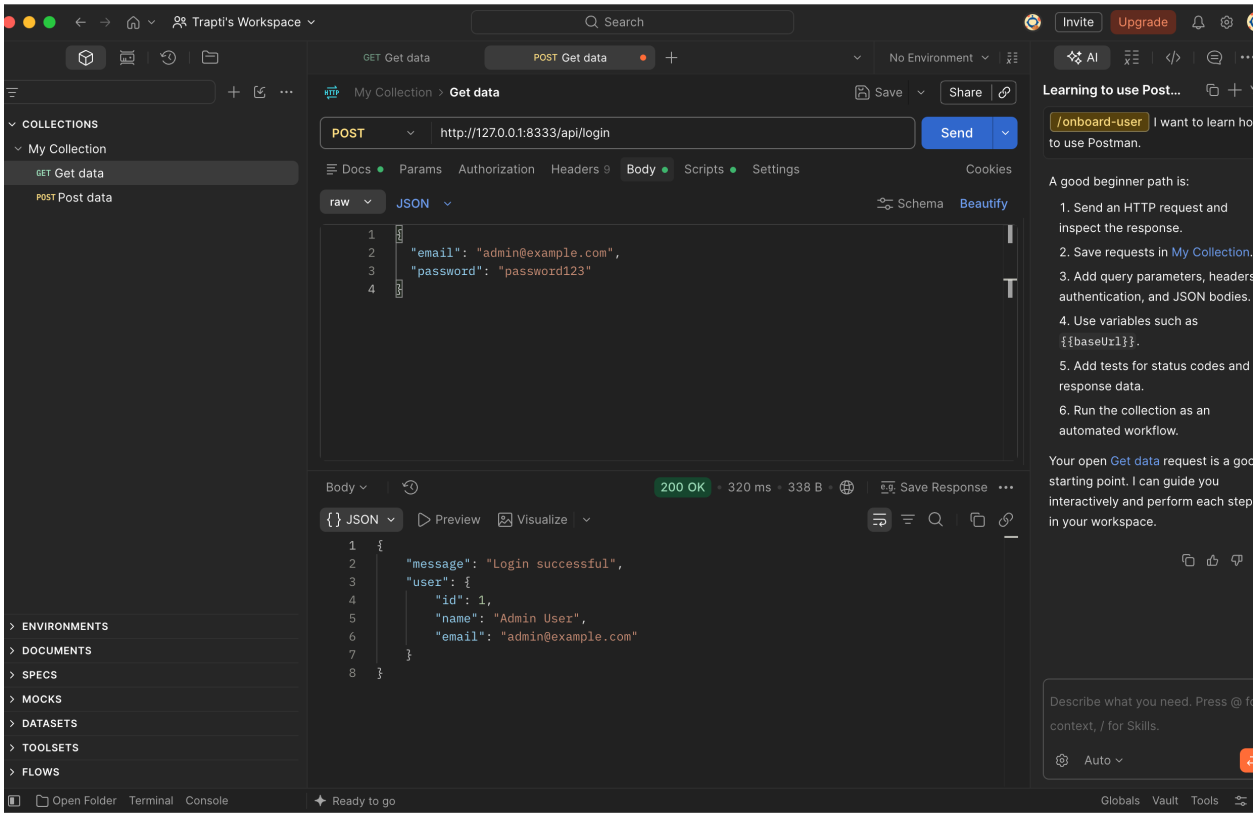
    return {
        "message": "Login successful",
        "user": {
            "id": user.id,
            "name": user.name,
            "email": user.email
        }
    }
```

```python
from passlib.context import CryptContext

pwd_context = CryptContext(
    schemes=["bcrypt"],
    deprecated="auto"
)


def hash_password(password: str) -> str:
    return pwd_context.hash(password)


def verify_password(
    plain_password: str,
    password_hash: str
) -> bool:
    return pwd_context.verify(
        plain_password,
        password_hash
    )
```



Screenshot: Successful login request in Postman.

Result / Observation:

Result / Observation: Valid credentials successfully created a server-side session in MySQL and returned an opaque session token through an HTTP-only cookie. This confirms that session information is maintained on the server rather than storing user information directly in the browser cookie.

2.7 Protected Routes / Session Validation:

I added a session-validation dependency that reads the opaque session_token from the HTTP-only cookie and verifies that the token exists in the MySQL sessions table. The session must exist, must not be expired, and must reference a valid user. Requests without a valid session are rejected with HTTP 401.

```python
def get_current_user(
    request: Request,
    db: Session = Depends(get_db)
):
    session_token = request.cookies.get("session_token")

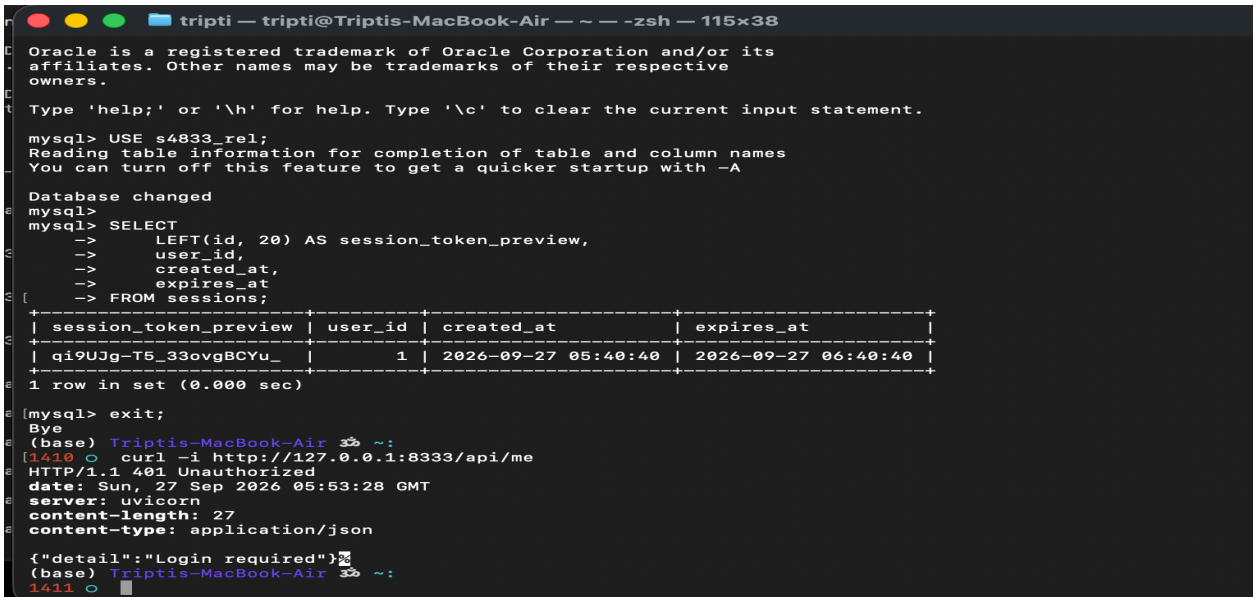
    if not session_token:
        raise HTTPException(
            status_code=401,
            detail="Login required"
        )

    session = (
        db.query(SessionDB)
        .filter(SessionDB.id == session_token)
        .first()
    )

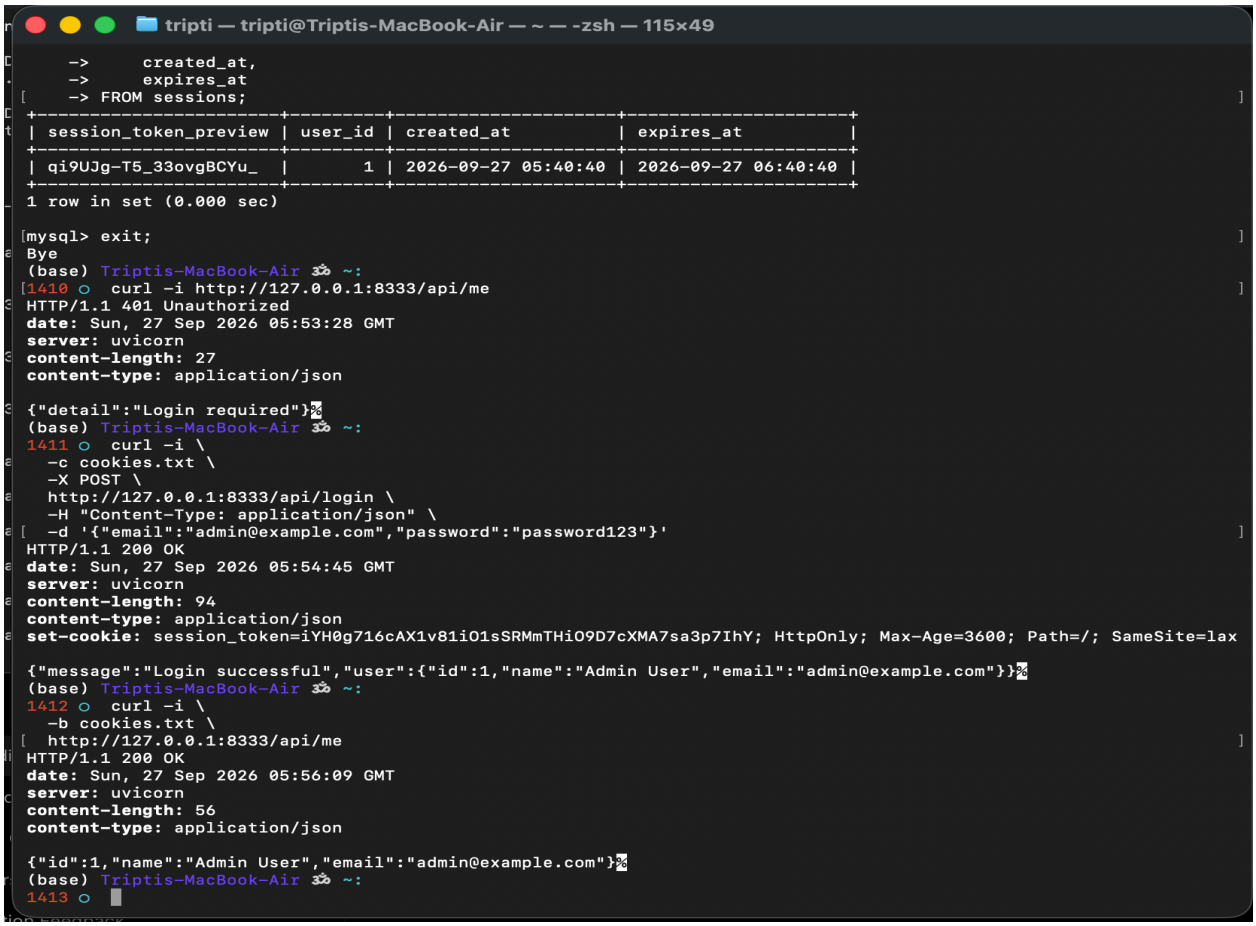
    if session is None:
        raise HTTPException(
            status_code=401,
            detail="Invalid session"
        )
```

```python

@router.get("/me")
def get_me(
    current_user: UserDB = Depends(get_current_user)
):
    return {
        "id": current_user.id,
        "name": current_user.name,
        "email": current_user.email
    }
```



Screenshot: Protected endpoint rejecting an unauthenticated request.



Screenshot: Protected endpoint successfully accessed using a valid server-side session.

Result / Observation:

Result / Observation: The protected endpoint correctly rejected requests without a session cookie and returned the authenticated user's information when a valid server-side session was supplied.

2.8 CRUD POST:

I implemented a protected POST endpoint to create new clinical trial records. The record is inserted into the MySQL clinical_trials table and MySQL generates the auto-incrementing ID.

```
@router.post(
    "",
    response_model=TrialResponse,
    status_code=201
)
def create_trial(
    trial: TrialCreate,
    db: Session = Depends(get_db),
    current_user: UserDB = Depends(get_current_user)
):
    new_trial = ClinicalTrialDB(
        trialTitle=trial.trialTitle.strip(),
        sponsor=trial.sponsor.strip()
    )

    db.add(new_trial)
    db.commit()
    db.refresh(new_trial)

    return new_trial
```

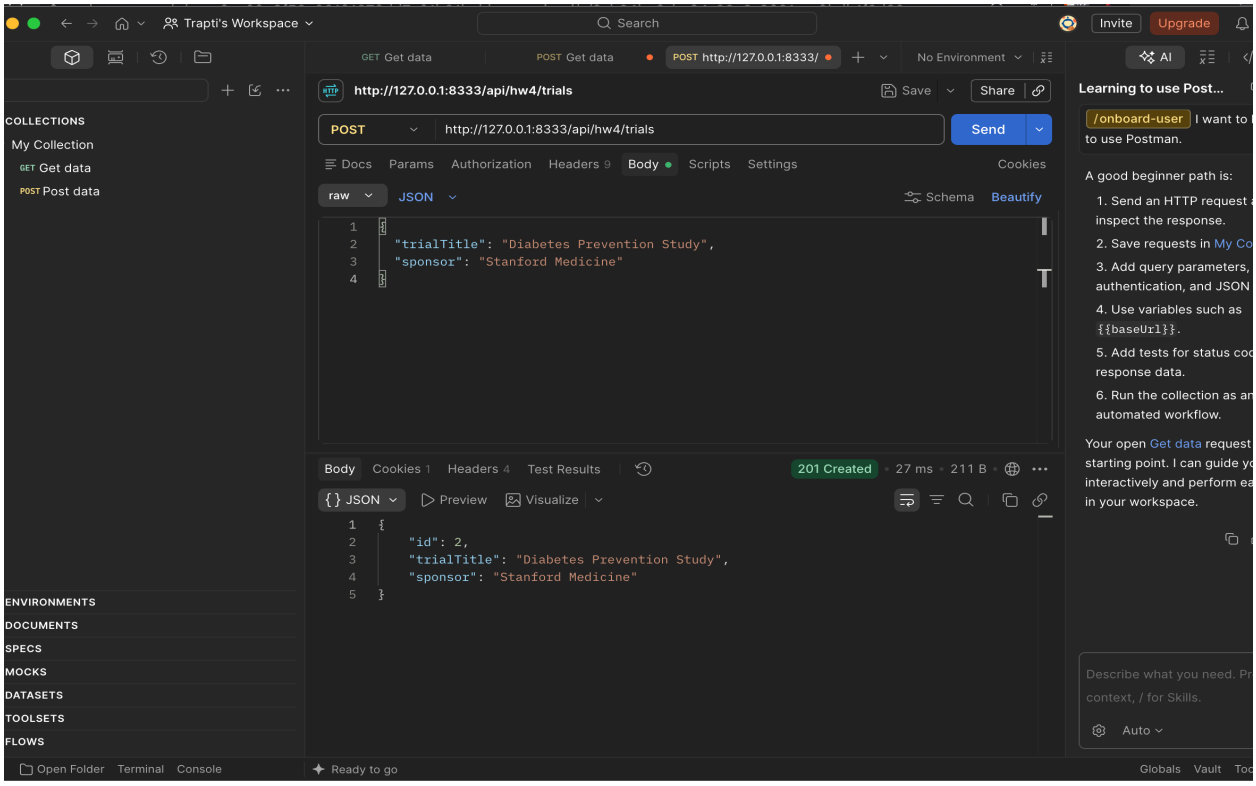
```
{"detail":"Login required"}%
(base) Triptis-MacBook-Air ~:
1414 ○  rm -f cookies.txt
(base) Triptis-MacBook-Air ~:
1415 ○  curl -i \
  -c cookies.txt \
  -X POST \
  http://127.0.0.1:8333/api/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@example.com","password":"password123"}'
HTTP/1.1 200 OK
date: Mon, 28 Sep 2026 04:25:26 GMT
server: uvicorn
content-length: 94
content-type: application/json
set-cookie: session_token=NQnqjuPWpGhXP1xBT0FClx3Tzu4HAEejAlYhaQVq82c; HttpOnly; Max-Age=3600; Path=/; SameSite=lax

{"message":"Login successful","user":{"id":1,"name":"Admin User","email":"admin@example.com"}}%
(base) Triptis-MacBook-Air ~:
1416 ○  cat cookies.txt
# Netscape HTTP Cookie File
# https://curl.se/docs/http-cookies.html
# This file was generated by libcurl! Edit at your own risk.

#HttpOnly_127.0.0.1     FALSE   /       FALSE   1790573126      session_token   NQnqjuPWpGhXP1xBT0FClx3Tzu4HAEejAlY
haQVq82c
(base) Triptis-MacBook-Air ~:
1417 ○  curl -i \
  -b cookies.txt \
  -X POST \
  http://127.0.0.1:8333/api/hw4/trials \
  -H "Content-Type: application/json" \
  -d '{"trialTitle":"Diabetes Prevention Study","sponsor":"Stanford Medicine"}'
HTTP/1.1 201 Created
date: Mon, 28 Sep 2026 04:26:08 GMT
server: uvicorn
content-length: 79
content-type: application/json

{"id":1,"trialTitle":"Diabetes Prevention Study","sponsor":"Stanford Medicine"}%
(base) Triptis-MacBook-Air ~:
1418 ○
```

Screenshot: Successful creation of a clinical trial using the POST endpoint.



Screenshot: Successful creation of a clinical trial using the POST endpoint in Postman.

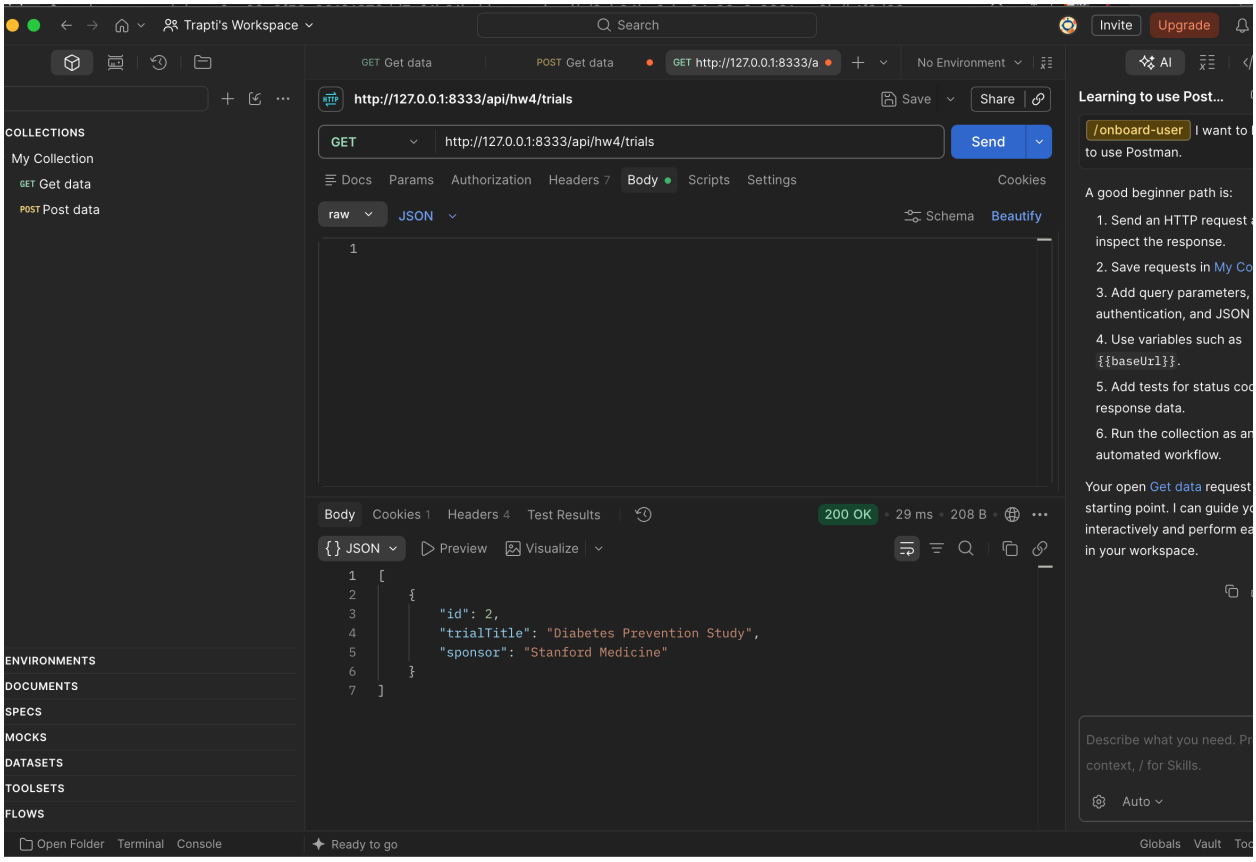
2.9 CRUD – GET All

I implemented a protected GET endpoint that retrieves all clinical trial records from the MySQL clinical_trials table. The records are returned in ascending order by ID.

```python

@router.get(
    "",
    response_model=list[TrialResponse]
)
def get_trials(
    db: Session = Depends(get_db),
    current_user: UserDB = Depends(get_current_user)
):
    return (
        db.query(ClinicalTrialDB)
        .order_by(ClinicalTrialDB.id)
        .all()
    )

```



Screenshot: GET endpoint returning all clinical trial records in Postman.

2.10 CRUD GET by ID:

I implemented a protected GET-by-ID endpoint that retrieves one clinical trial using its unique ID. If the requested ID does not exist, the endpoint returns a 404 response.

```python
@router.get(
    "/{trial_id}",
    response_model=TrialResponse
)
def get_trial(
    trial_id: int,
    db: Session = Depends(get_db),
    current_user: UserDB = Depends(get_current_user)
):
    trial = db.get(ClinicalTrialDB, trial_id)

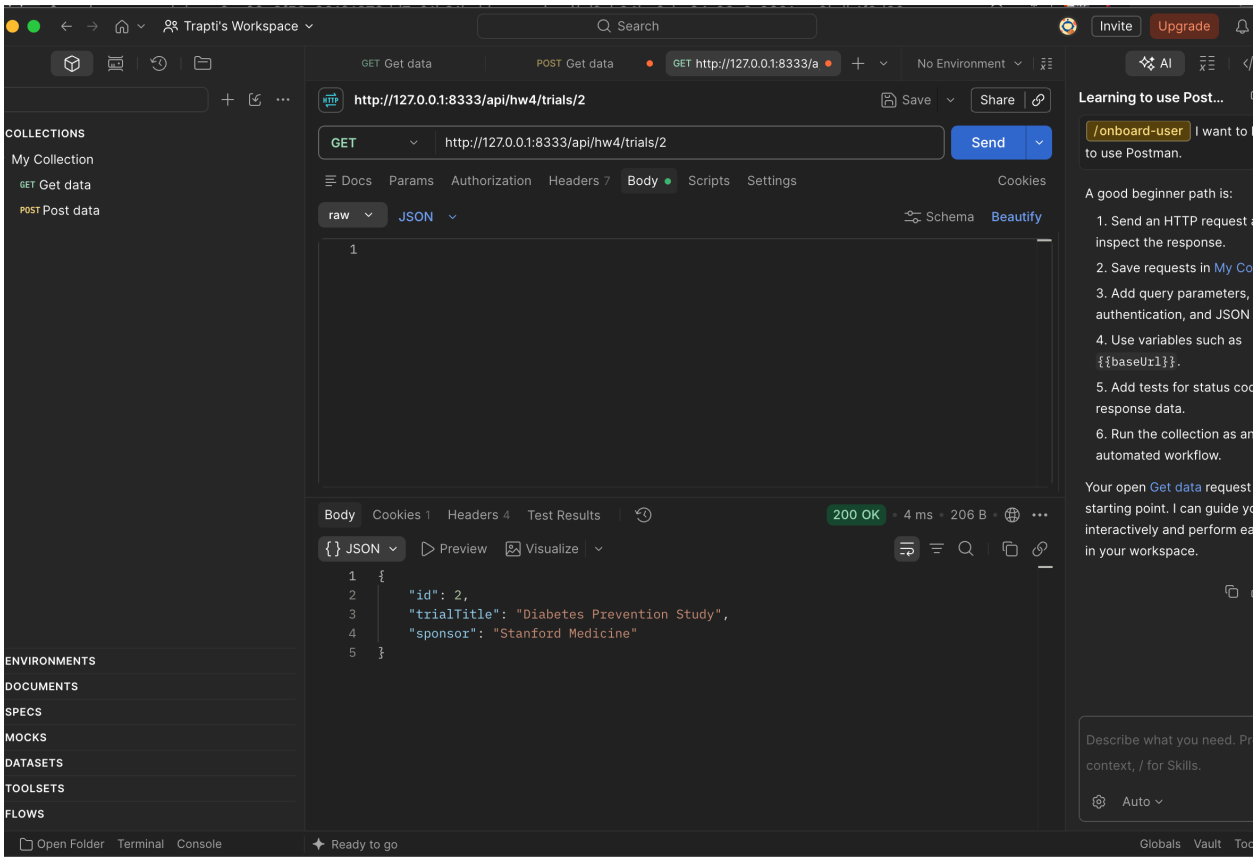
    if trial is None:
        raise HTTPException(
            status_code=404,
            detail="Clinical trial not found"
        )

    return trial
```

```
(base) Triptis-MacBook-Air ॐ ~:
1418 ○  curl -i \
  -b cookies.txt \
  http://127.0.0.1:8333/api/hw4/trials
HTTP/1.1 200 OK
date: Mon, 28 Sep 2026 04:29:54 GMT
server: uvicorn
content-length: 81
content-type: application/json

[{"id":1,"trialTitle":"Diabetes Prevention Study","sponsor":"Stanford Medicine"}]%
(base) Triptis-MacBook-Air ॐ ~:
1419 ○
```

Screenshot: GET-by-ID endpoint returning a specific clinical trial.



Screenshot: GET-by-ID endpoint returning a specific clinical trial in Postman.

2.11 CRUD PUT:

I implemented a protected PUT endpoint that updates the trial title and sponsor of an existing clinical trial. The updated values are committed to MySQL and returned in the response.

```
    trial.trialTitle = trial_data.trialTitle.strip()
    trial.sponsor = trial_data.sponsor.strip()

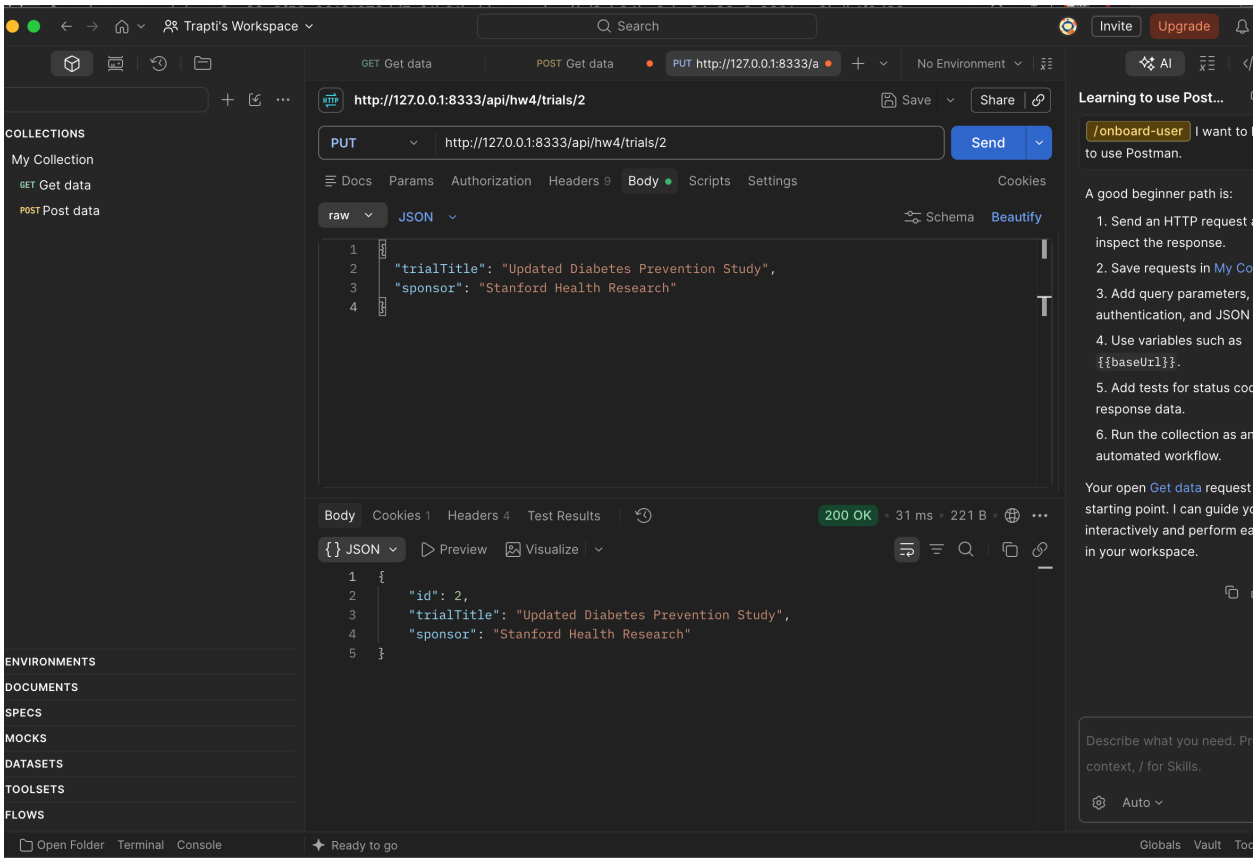
    db.commit()
    db.refresh(trial)

    return trial
```

```
(base) Triptis-MacBook-Air ॐ ~:
1419 ○  curl -i \
  -b cookies.txt \
  -X PUT \
  http://127.0.0.1:8333/api/hw4/trials/1 \
  -H "Content-Type: application/json" \
  -d '{"trialTitle":"Updated Diabetes Prevention Study","sponsor":"Stanford Health Research"}'
HTTP/1.1 200 OK
date: Mon, 28 Sep 2026 04:33:21 GMT
server: uvicorn
content-length: 94
content-type: application/json

{"id":1,"trialTitle":"Updated Diabetes Prevention Study","sponsor":"Stanford Health Research"}%
(base) Triptis-MacBook-Air ॐ ~:
1420 ○
```

Screenshot: Successful update of a clinical trial using the PUT endpoint.



Screenshot: Successful update of a clinical trial using the PUT endpoint in Postman.

2.12 CRUD DELETE:

I implemented a protected DELETE endpoint that removes the selected clinical trial from the MySQL database and commits the deletion.

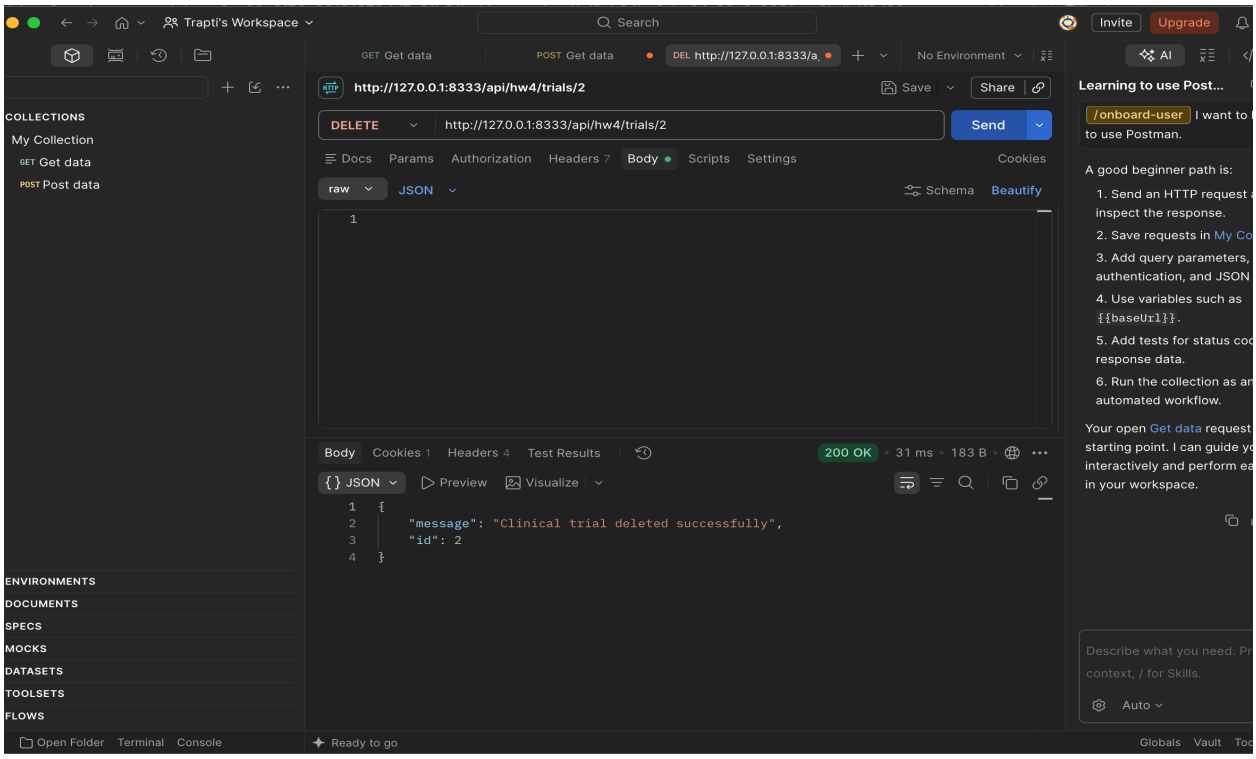
```
db.delete(trial)
db.commit()

return {
    "message": "Clinical trial deleted successfully",
    "id": trial_id
}
```

```
(base) Triptis-MacBook-Air ॐ ~:
1420 o  curl -i \
  -b cookies.txt \
  -X DELETE \
  http://127.0.0.1:8333/api/hw4/trials/1
HTTP/1.1 200 OK
date: Mon, 28 Sep 2026 04:34:40 GMT
server: uvicorn
content-length: 56
content-type: application/json

{"message":"Clinical trial deleted successfully","id":1}%
(base) Triptis-MacBook-Air ॐ ~:
1421 o
```

Screenshot: Successful deletion of a clinical trial using the DELETE endpoint.

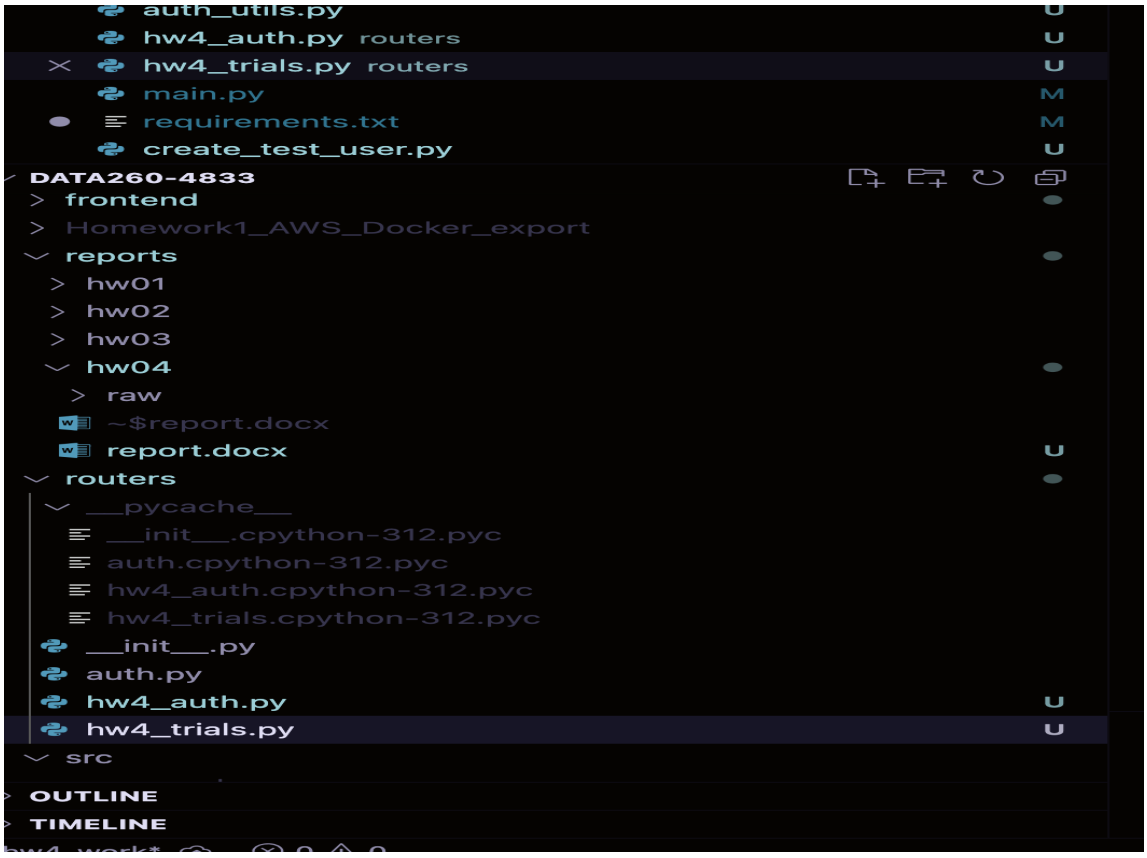


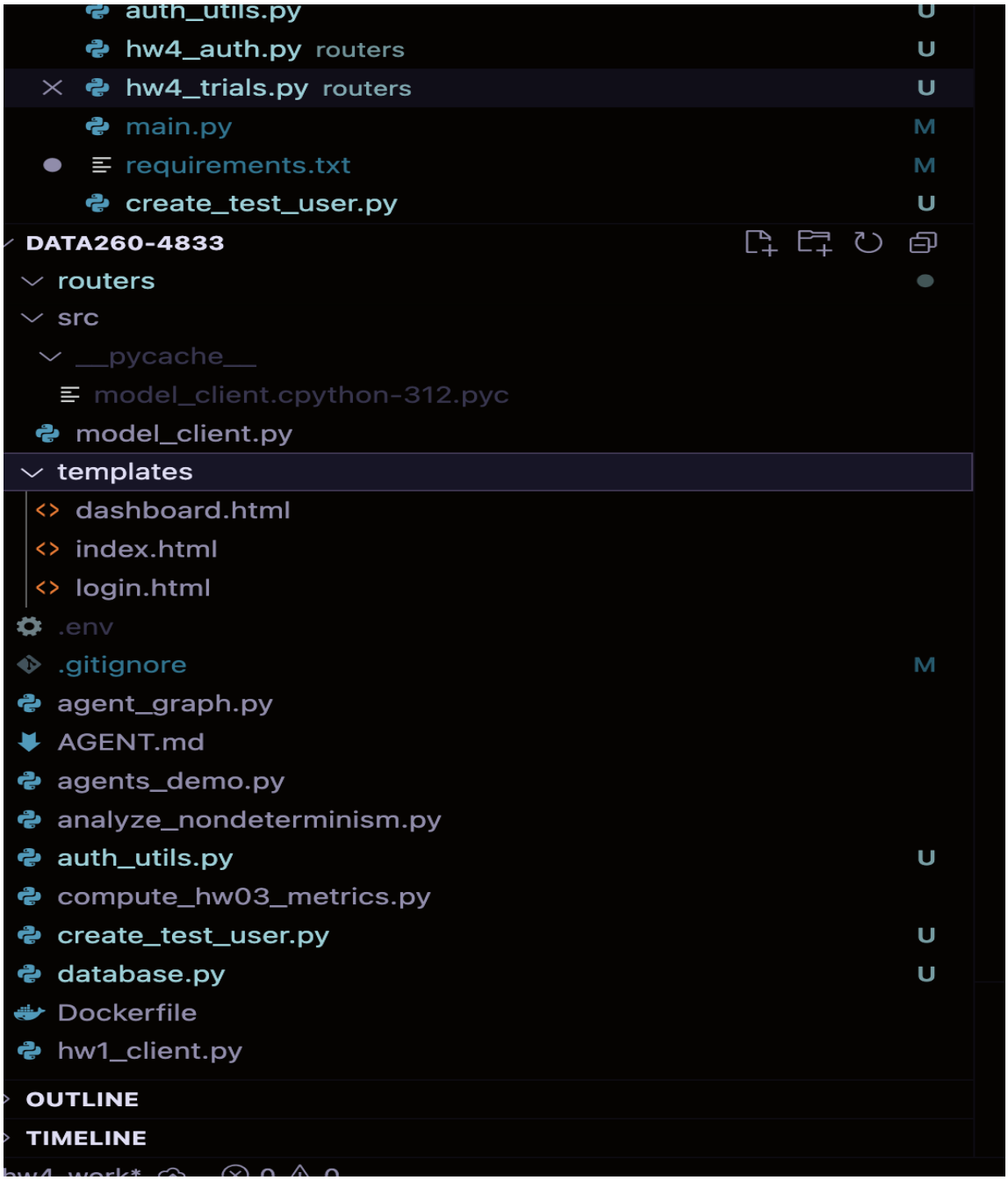
Screenshot: Successful deletion of a clinical trial using the DELETE endpoint in Postman.

Result / Observation: The FastAPI backend successfully performed Create, Read, Update, and Delete operations against the MySQL clinical_trials table. Each operation required a valid authenticated session.

2.13 Project Folder Structure:

The project continues to extend the existing data260-4833 repository. The React client is stored under frontend/, while the FastAPI authentication and CRUD routes are organized under routers/. Database connection and SQLAlchemy models remain in shared root-level files.





PART 3:

3.1 Seed Data:

I created a reproducible seed script for the Part 3 performance experiment using SEED = 4833. The script generates 5,000 records in the primary clinical_trials table and 200

records in the related trial_notes table. The related rows are associated with existing clinical trial IDs so they can be used to demonstrate the N+1 query problem. The fixed seed ensures that the generated dataset can be reproduced consistently.



```python
import random

from database import db_session_basede26
from models import ClinicalTrialDB, TrialNoteDB


SEED = 4833
PRIMARY_ROWS = 5000
RELATED_ROWS = 200

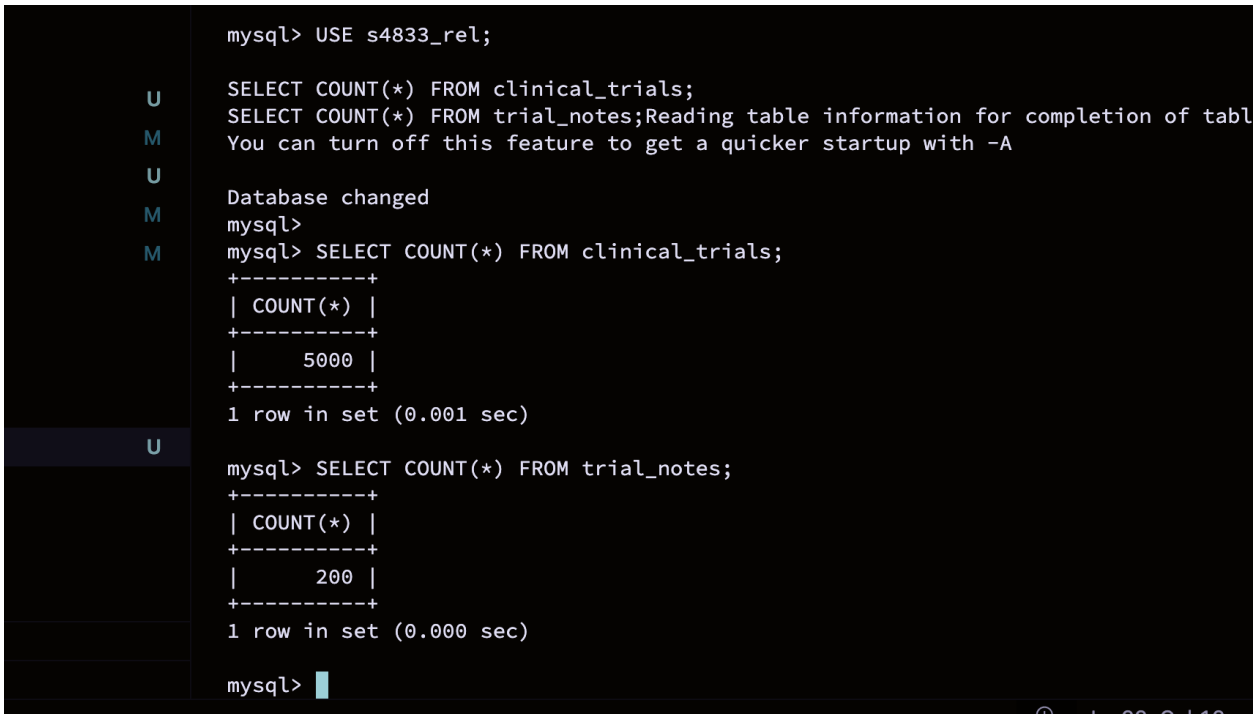

def seed_data():
    random.seed(SEED)

    db = db_session_basede26()

    try:
        db.query(TrialNoteDB).delete()
        db.query(ClinicalTrialDB).delete()
        db.commit()

        trials = []
```

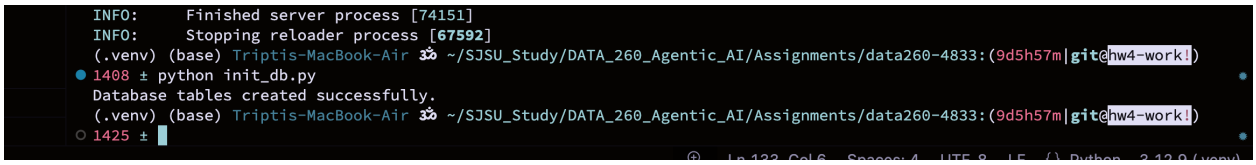
```
(.venv) (base) Triptis-MacBook-Air ~/SJSU_Study/DATA_260_Agentic_AI/Assignment
1426 ± python seed_hw04.py
Seed: 4833
Primary rows: 5000
Related rows: 200
(.venv) (base) Triptis-MacBook-Air ~/SJSU_Study/DATA_260_Agentic_AI/Assignment
1427 ±
```



Screenshot: seed script output and MySQL count verification

3.2 Related Table:

I created a temporary related table named trial_notes for the N+1 experiment. Each row contains a foreign key trial_id that links the related data to a record in the clinical_trials table. This related table is used only for the Part 3 performance experiment.



```python
class TrialNoteDB(Base):
    __tablename__ = "trial_notes"

    id = Column(
        Integer,
        primary_key=True,
        autoincrement=True
    )

    trial_id = Column(
        Integer,
        ForeignKey("clinical_trials.id"),
        nullable=False
    )

    note = Column(
        String(255),
        nullable=False
    )
```

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE s4833_rel;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW TABLES;
+---------------------+
| Tables_in_s4833_rel |
+---------------------+
| clinical_trials     |
| sessions            |
| trial_notes         |
| users               |
+---------------------+
4 rows in set (0.001 sec)

mysql> DESCRIBE trial_notes;
+----------+--------------+------+-----+---------+----------------+
| Field    | Type         | Null | Key | Default | Extra          |
+----------+--------------+------+-----+---------+----------------+
| id       | int          | NO   | PRI | NULL    | auto_increment |
| trial_id | int          | NO   | MUL | NULL    |                |
| note     | varchar(255) | NO   |     | NULL    |                |
+----------+--------------+------+-----+---------+----------------+
3 rows in set (0.001 sec)

mysql>
```

Screenshot: MySQL verification of the trial_notes related table and its schema.

3.3 Naive N+1 Endpoint:

I created an intentionally naive list endpoint for demonstrating the N+1 query problem. The endpoint first retrieves the requested number of clinical trial records. It then loops through those records and executes a separate SQL query for each clinical trial to retrieve its related notes. Therefore, a request returning N clinical trials requires one query for the trial list plus N additional related-data queries.

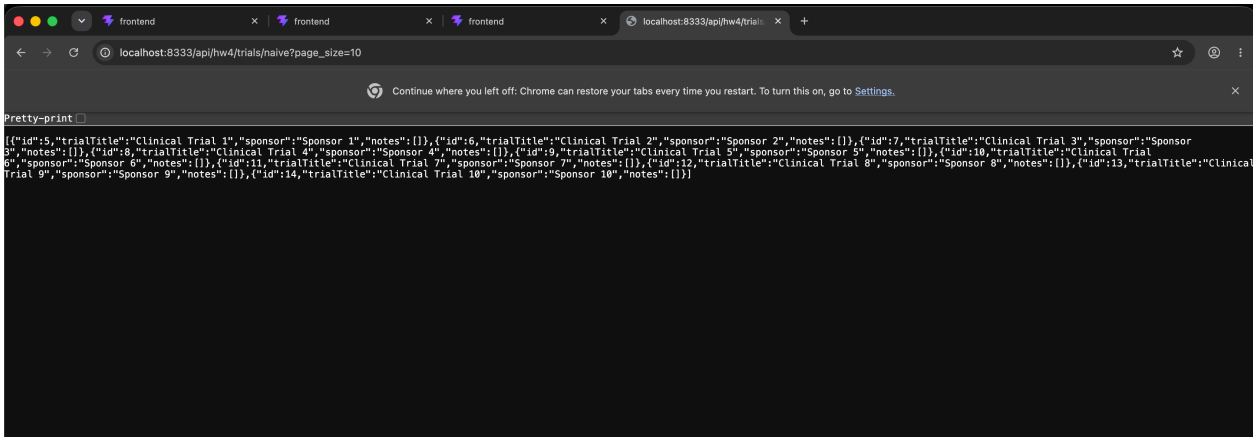
routers > hw4_trials.py > delete_trial

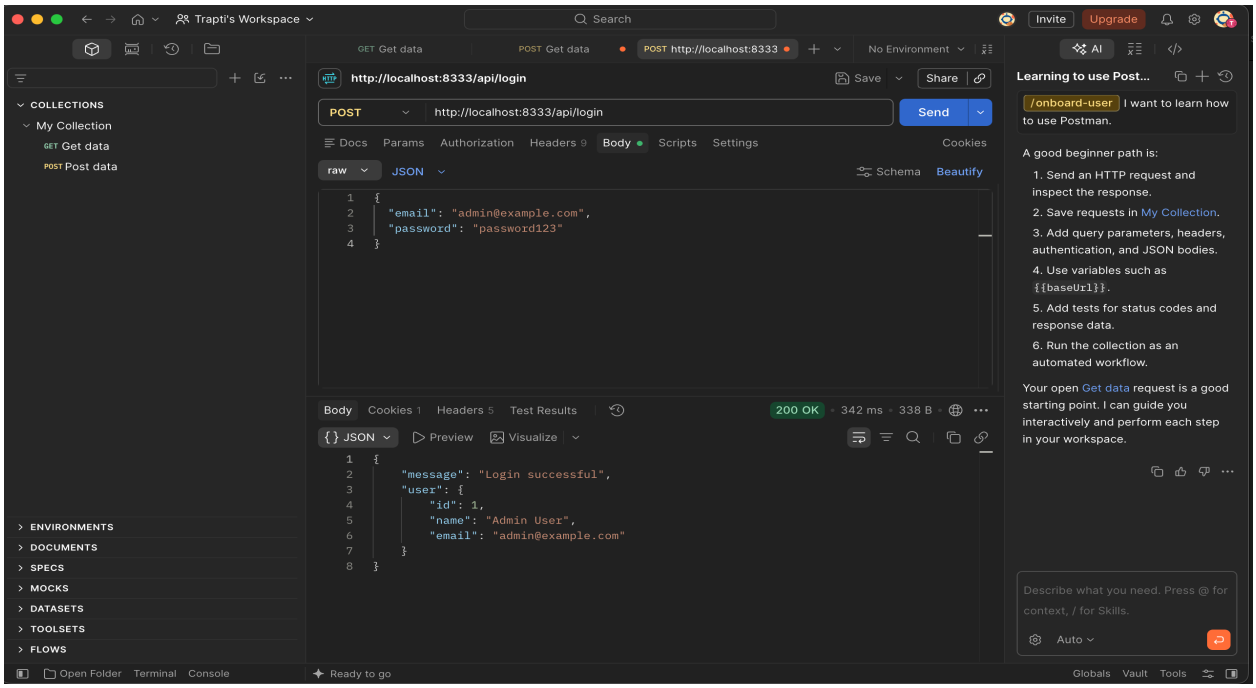
```python
def get_trials_naive(
    current_user: UserDB = Depends(get_current_user)
):
    trials = (
        db.query(ClinicalTrialDB)
        .order_by(ClinicalTrialDB.id)
        .limit(page_size)
        .all()
    )

    results = []

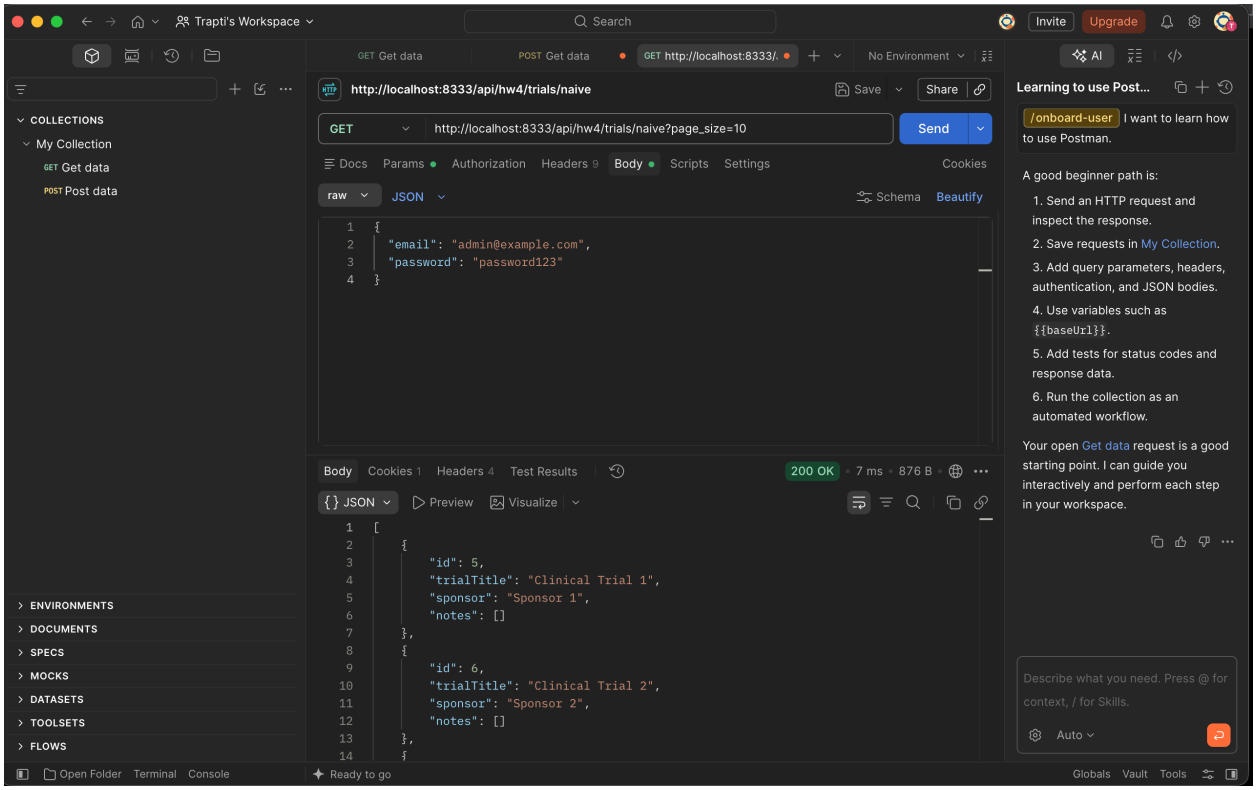
    for trial in trials:
        notes = (
            db.query(TrialNoteDB)
            .filter(TrialNoteDB.trial_id == trial.id)
            .all()
        )

        results.append(
            {
                "id": trial.id,
                "trialTitle": trial.trialTitle,
                "sponsor": trial.sponsor,
                "notes": notes
            }
```

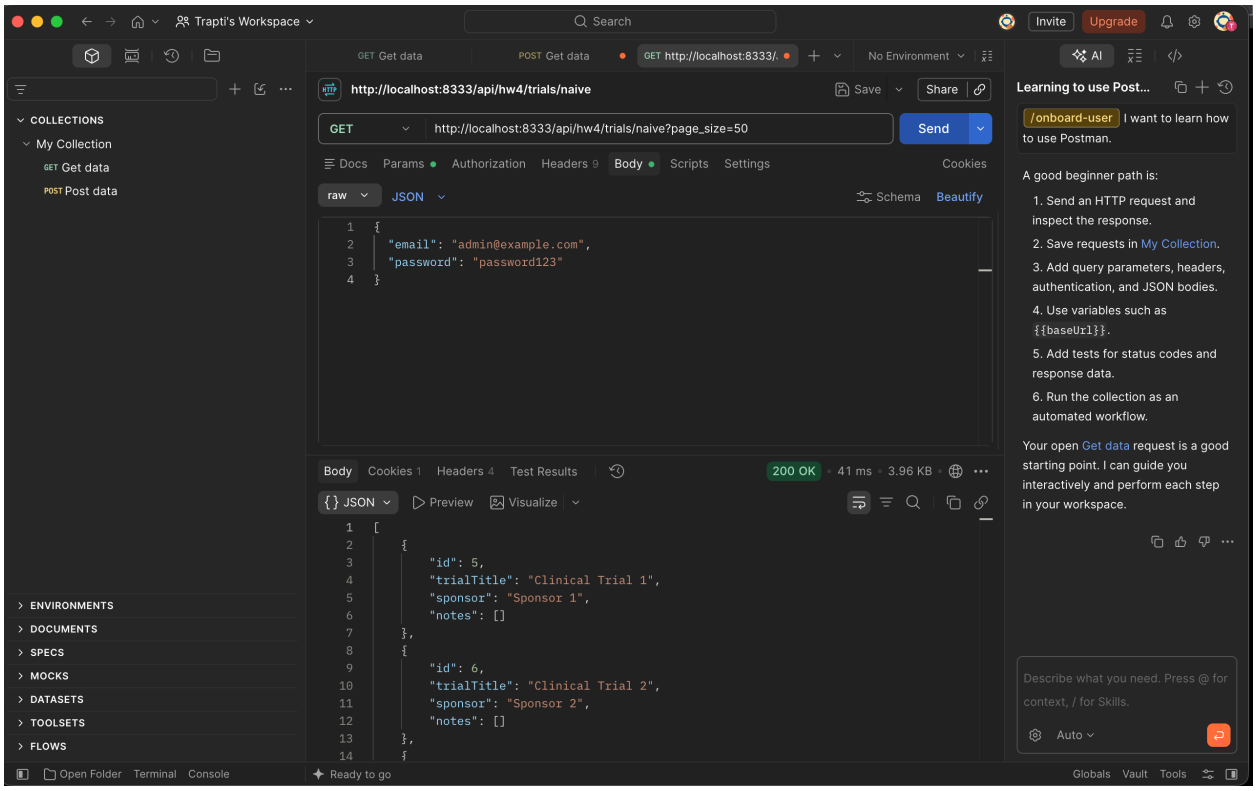




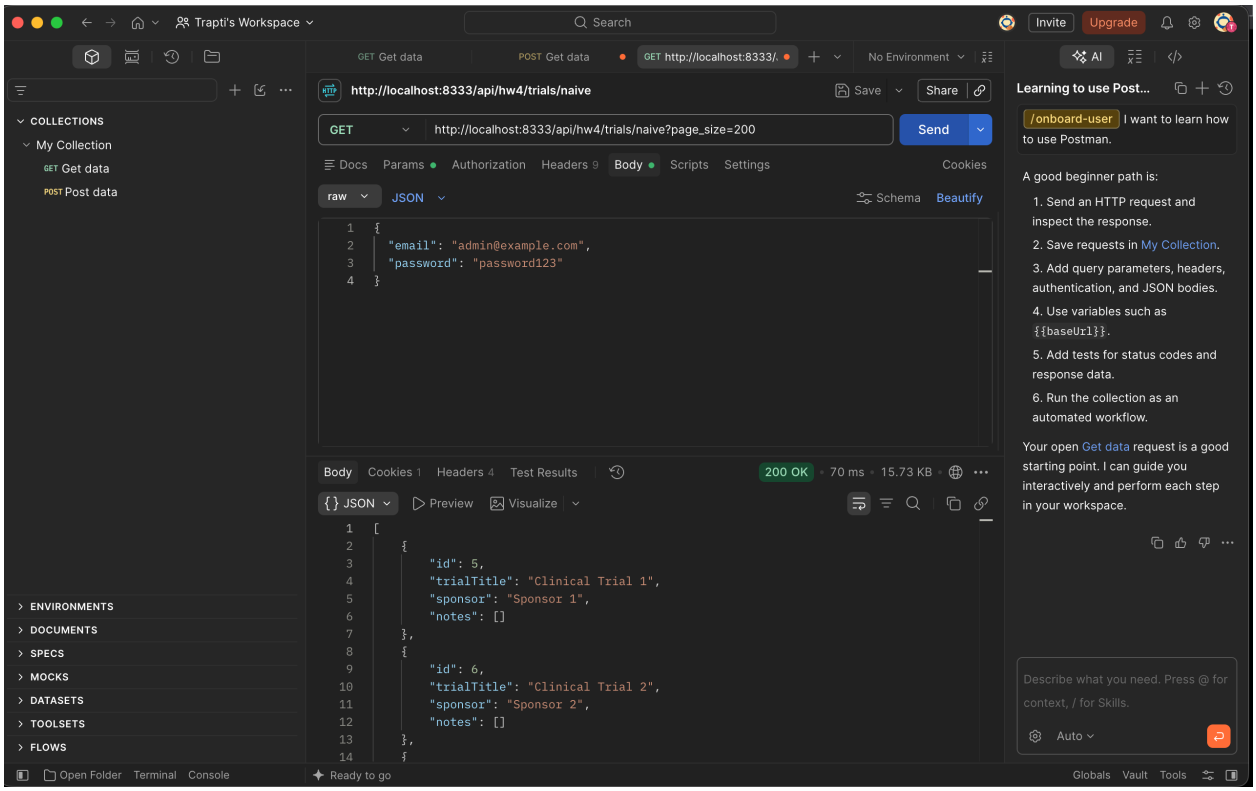
Screenshot: Login_succesful



Screenshot: Naive endpoint working with page_size=10



Screenshot: Naive endpoint working with page_size=50

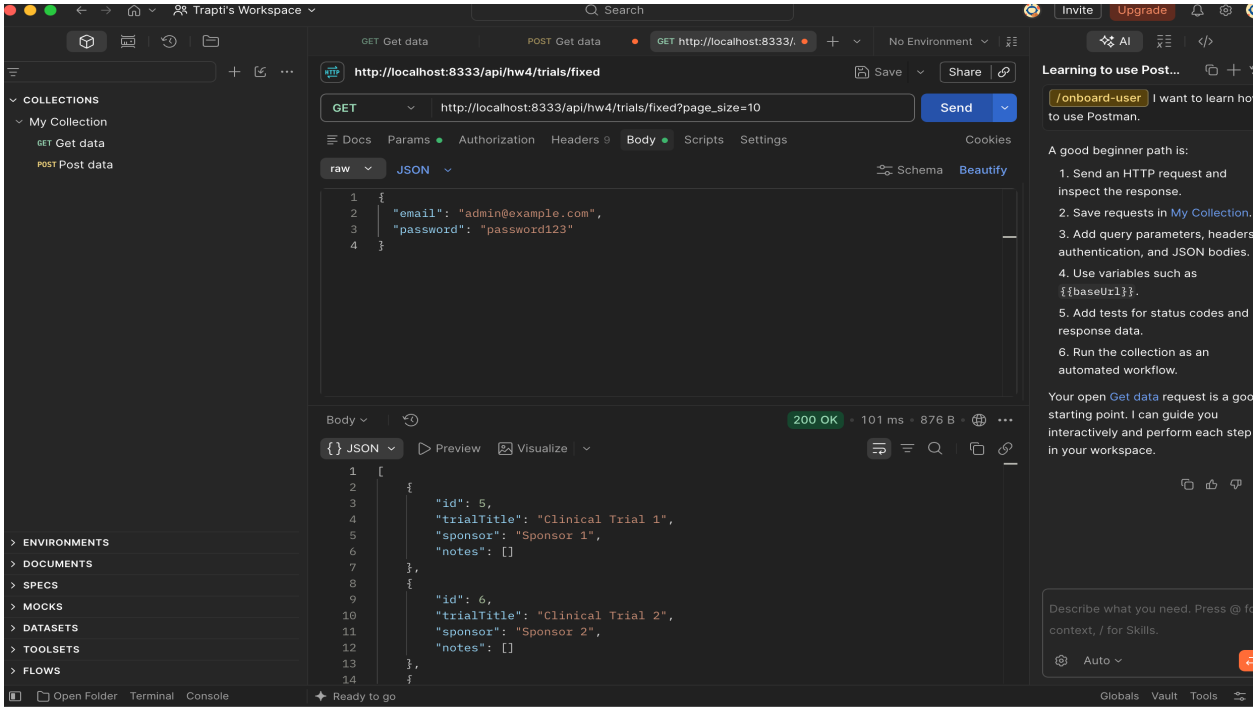


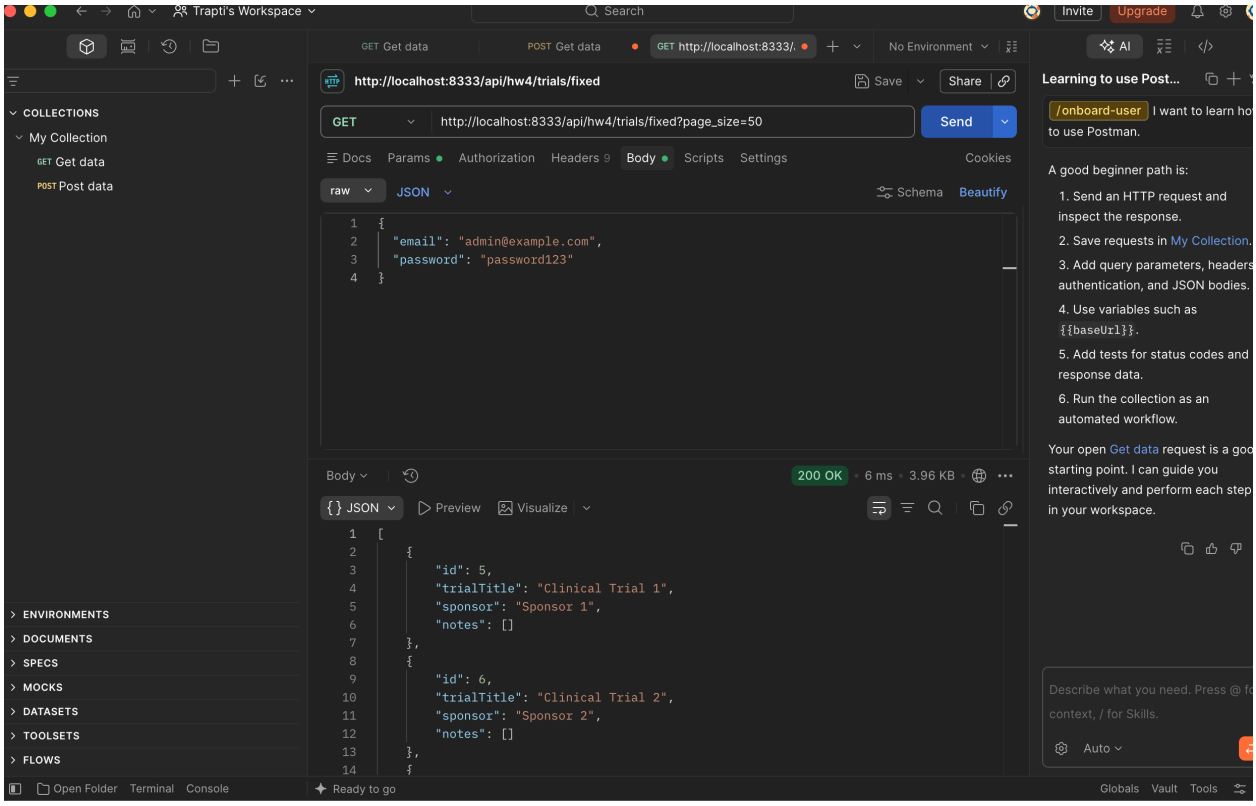
Screenshot: Naive endpoint working with page_size=200

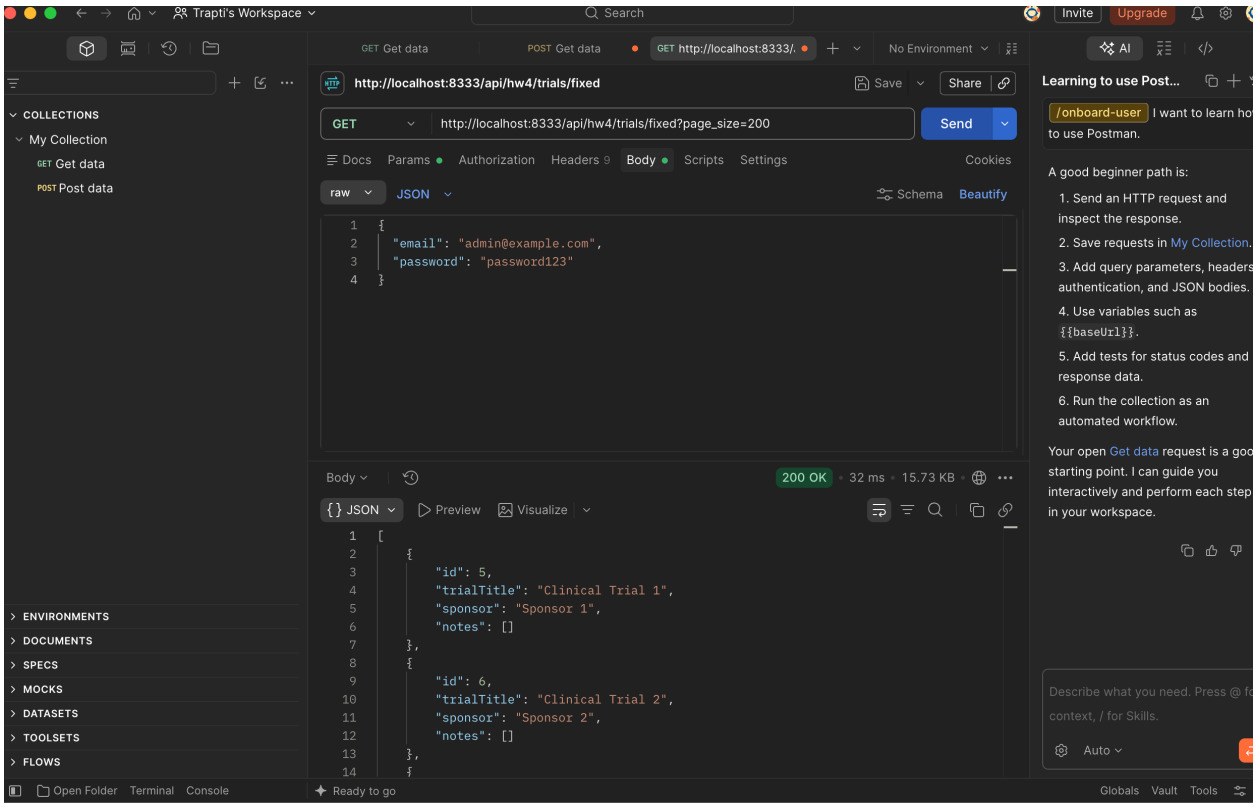
3.4 Fixed Endpoint:

I fixed the N+1 query problem using SQLAlchemy eager loading with selectinload. Instead of executing a separate query for every clinical trial, the fixed endpoint loads the requested clinical trials and retrieves their related notes in a separate bulk query. This reduces the number of SQL queries significantly as the page size increases.





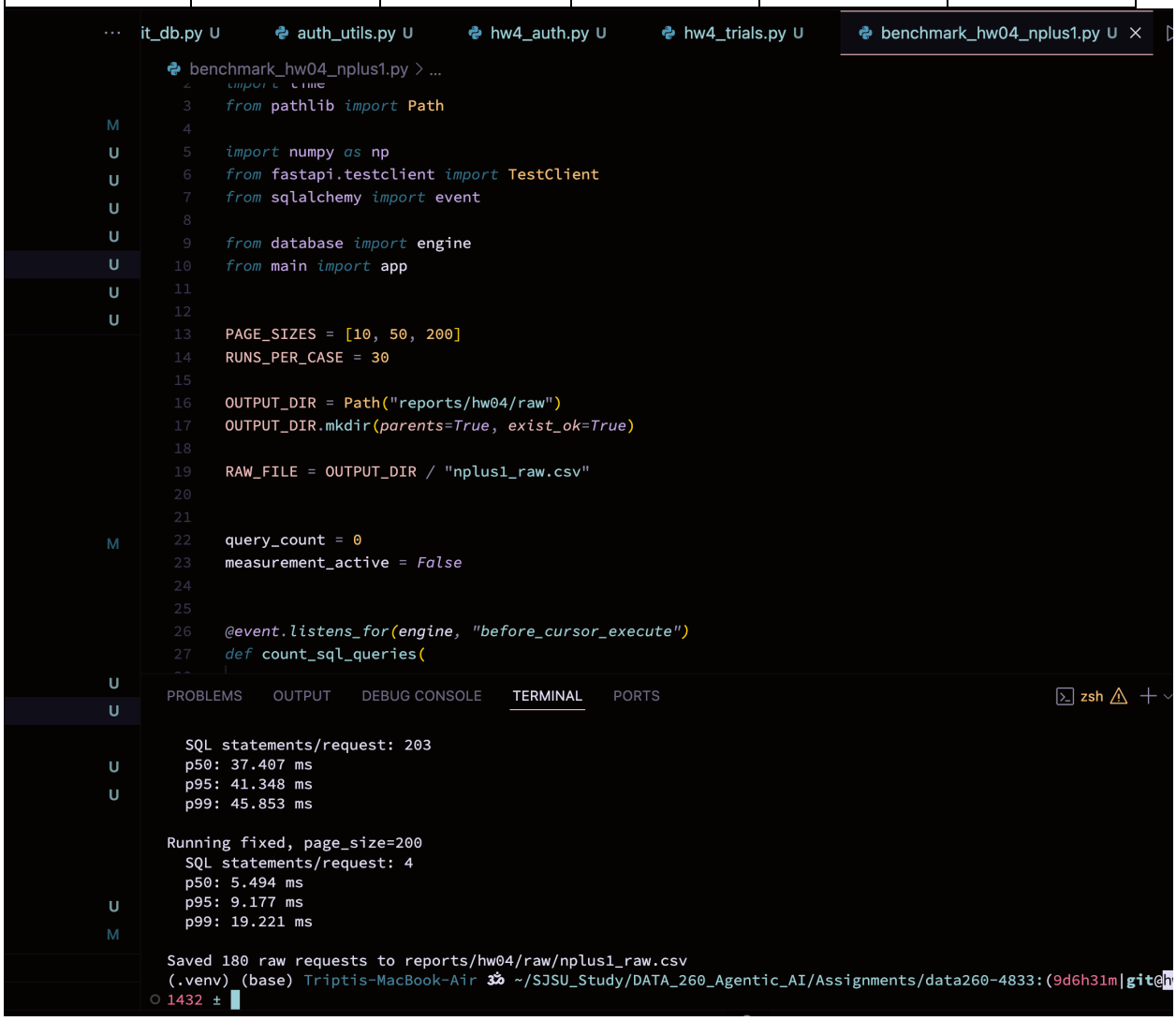

Trapti's Workspace
Search
Invite
Upgrade
GET Get data
POST Get data
GET http://localhost:8333/.
No Environment
AI
COLLECTIONS
My Collection
GET Get data
POST Post data
http://localhost:8333/api/hw4/trials/fixed
Save
Share
Learning to use Post...
GET
http://localhost:8333/api/hw4/trials/fixed?page_size=50
Send
/onboard-user I want to learn ho
to use Postman.
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
A good beginner path is:
1. Send an HTTP request and inspect the response.
2. Save requests in My Collection.
3. Add query parameters, headers authentication, and JSON bodies.
4. Use variables such as {{baseUrl}}.
5. Add tests for status codes and response data.
6. Run the collection as an automated workflow.
Your open Get data request is a goo
starting point. I can guide you
interactively and perform each step
in your workspace.
Body
200 OK
6 ms
3.96 KB
{} JSON
Preview
Visualize
Describe what you need. Press @ fo
context, / for Skills.
Auto
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
TOOLSETS
FLOWS
Open Folder
Terminal
Console
Ready to go
Globals
Vault
Tools


Trapti's Workspace
Search
Invite
Upgrade
GET Get data
POST Get data
GET http://localhost:8333/.
No Environment
AI
COLLECTIONS
My Collection
GET Get data
POST Post data
http://localhost:8333/api/hw4/trials/fixed
Save
Share
Learning to use Post...
GET
http://localhost:8333/api/hw4/trials/fixed?page_size=200
Send
/onboard-user I want to learn ho
to use Postman.
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
A good beginner path is:
1. Send an HTTP request and inspect the response.
2. Save requests in My Collection.
3. Add query parameters, headers authentication, and JSON bodies.
4. Use variables such as {{baseUrl}}.
5. Add tests for status codes and response data.
6. Run the collection as an automated workflow.
Your open Get data request is a goo
starting point. I can guide you
interactively and perform each step
in your workspace.
Body
200 OK
32 ms
15.73 KB
{} JSON
Preview
Visualize
Describe what you need. Press @ fo
context, / for Skills.
Auto
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
TOOLSETS
FLOWS
Open Folder
Terminal
Console
Ready to go
Globals
Vault
Tools

3.5 Benchmark Script / 180 requests:

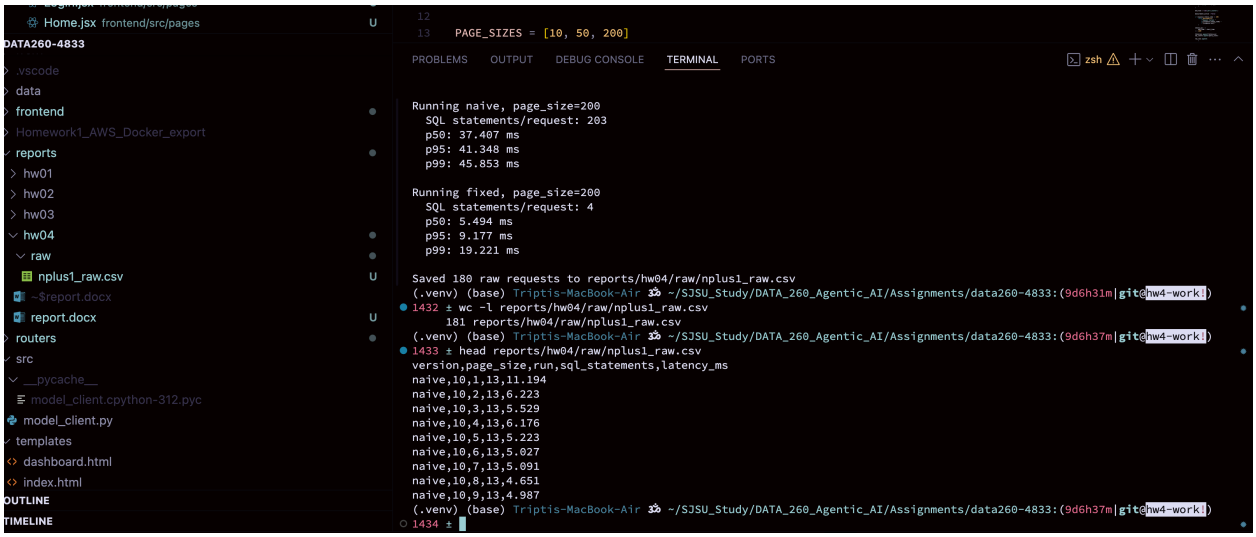
I benchmarked both the naive and fixed list endpoints using page sizes of 10, 50, and 200. Each endpoint/page-size combination was executed 30 times, producing a total of 180 measured requests. For every request, I recorded the number of SQL statements and request latency. The fixed endpoint used eager loading with selectinload, which kept the SQL statement count constant as the page size increased.

Page Size	Version	SQL stmts/req	p50 (ms)	p95 (ms)	p99 (ms)
10	naive	13	4.964	6.202	9.752
10	fixed	4	2.950	4.836	14.870
50	naive	53	11.085	13.183	13.850
50	fixed	4	3.096	3.541	3.816
200	naive	203	37.407	41.348	45.853
200	fixed	4	5.494	9.177	19.221



3.6 Raw Results:

I saved the raw benchmark output for all 180 measured requests in reports/hw04/raw/nplus1_raw.csv. The CSV contains one row for each request, including the endpoint version, page size, run number, SQL statement count, and latency in milliseconds. I verified the file contains 181 lines total, consisting of 1 header row and 180 measured request rows.



Screenshot: Verification of the raw N+1 benchmark CSV containing 180 measured requests plus the header row.

3.7 N+1 Results and Speed-up Analysis:

Page Size	Version	SQL stmts/req	p50 (ms)	p95 (ms)	p99 (ms)
10	naive	13	4.964	6.202	9.752
10	fixed	4	2.950	4.836	14.870
50	naive	53	11.085	13.183	13.850
50	fixed	4	3.096	3.541	3.816
200	naive	203	37.407	41.348	45.853
200	fixed	4	5.494	9.177	19.221

The fixed endpoint became increasingly faster as the page size grew. At page size 10, the fixed endpoint was approximately 1.68× faster based on median latency. At page size 50, it was approximately 3.58× faster, and at page size 200, it was approximately 6.81× faster.

The SQL statement count explains this difference. The naive endpoint increased from 13 statements at page size 10 to 203 statements at page size 200 because it executed an additional query for each clinical trial to retrieve related notes. In contrast, the fixed endpoint consistently used only 4 SQL statements per request because selectinload retrieved the related notes in a bulk query.

The fixed endpoint therefore scales much better as the number of returned records increases. The p99 latency for the fixed page-size-10 case was higher than its p50 and p95 values because one or more benchmark runs were slower outliers, but the overall trend still clearly shows the benefit of eager loading.

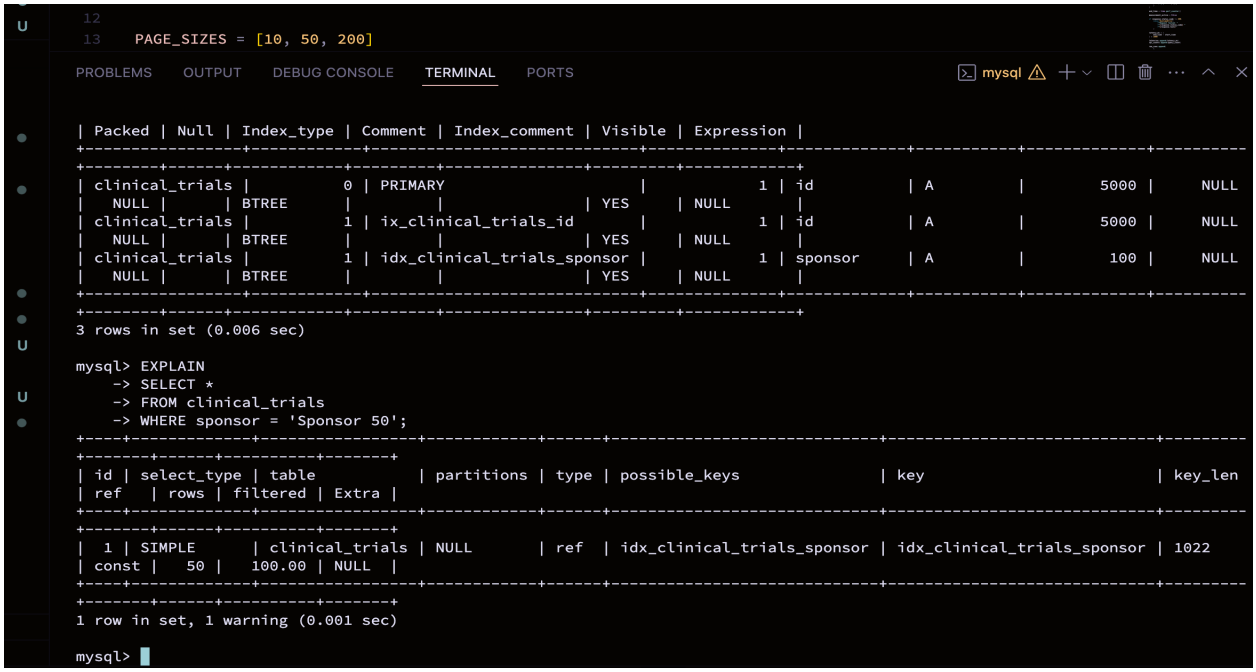
Page Size	p50 Speed-up	Median Latency Reduction
10	1.68×	40.6%
50	3.58×	72.1%
200	6.81×	85.3%

3.8 Database Index + EXPLAIN

I evaluated a lookup on the sponsor field before and after adding a database index. Before the index was created, MySQL had no dedicated index available for filtering clinical trials by sponsor, so the query required a broader scan of the table. I then created the idx_clinical_trials_sponsor index on the sponsor column and repeated the same EXPLAIN query.

After the index was added, MySQL was able to use the sponsor index to locate matching records more efficiently instead of scanning the full table. This demonstrates how an index can reduce the amount of data examined for selective lookup queries.





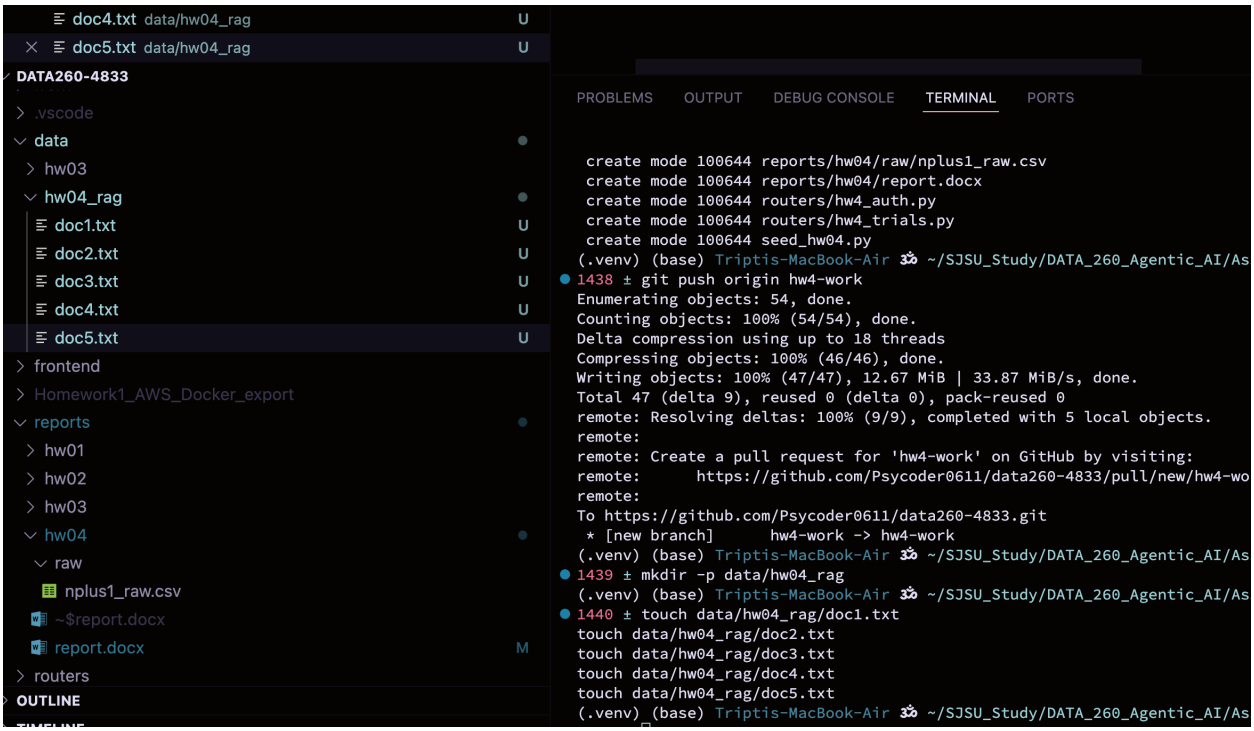
PART 4

4.1: Create the corpus:

I created a five-document clinical research corpus for the RAG experiment. The documents cover clinical trial phases, randomization and control groups, informed

consent, blinding, and eligibility/adverse events. The corpus was intentionally organized into related but distinct topics so retrieval quality, overlapping evidence, ambiguous questions, and refusal behavior could be evaluated consistently.

Document	Topic
doc1.txt	Clinical trial phases
doc2.txt	Randomization and control groups
doc3.txt	Informed consent
doc4.txt	Blinding in clinical research
doc5.txt	Eligibility criteria and adverse events



4.2 Chunking:

The five source documents were split into chunks using an initial chunk size of 500 characters and an overlap of 50 characters. Each chunk stores its text, source filename, and a unique chunk ID.

```python
def chunk_text(text, source):
    chunks = []

    start = 0
    chunk_number = 0

    while start < len(text):
        end = start + CHUNK_SIZE

        chunk = text[start:end].strip()

        if chunk:
            chunks.append(
                {
                    "text": chunk,
                    "source": source,
                    "chunk_id": (
                        f"{source}_chunk_{chunk_number}"
                    )
                }
            )

        chunk_number += 1

        if end >= len(text):
            break

        start = end - CHUNK_OVERLAP

    return chunks
```

```
1444 ± python rag.py
Documents loaded: 5
Chunks created: 7
Chunk size: 500
Chunk overlap: 50
```

4.3 Embeddings and Vector Store:

I generated local embeddings using the nomic-embed-text model through Ollama and stored the resulting vectors in a FAISS vector index. The vectors were normalized so cosine-style similarity could be used for retrieval.

```python
def build_vector_store(chunks):
    embeddings_model = OllamaEmbeddings(
        model=EMBED_MODEL
    )

    texts = [
        chunk["text"]
        for chunk in chunks
    ]

    embeddings = (
        embeddings_model.embed_documents(texts)
    )

    vectors = np.array(
        embeddings,
        dtype="float32"
    )

    faiss.normalize_L2(vectors)

    index = faiss.IndexFlatIP(
        vectors.shape[1]
    )

    index.add(vectors)

    return index, embeddings_model
```

```
FAISS vectors stored: 7
```

4.4 Retrieval:

For each question, the system converts the query into an embedding and retrieves the top 3 most similar chunks from FAISS. Before any LLM response is generated, the retrieved source, chunk ID, similarity score, and chunk text are printed so retrieval errors can be distinguished from generation errors.

```python
def retrieve(
    question,
    chunks,
    index,
    embeddings_model,
    top_k=TOP_K
):
    query_embedding = (
        embeddings_model.embed_query(question)
    )

    query_vector = np.array(
        [query_embedding],
        dtype="float32"
    )

    faiss.normalize_L2(query_vector)

    scores, indices = index.search(
        query_vector,
        top_k
    )

    results = []

    for score, index_position in zip(
        scores[0],
        indices[0]
    ):
        chunk = chunks[index_position]

        results.append(
            {
                "source": chunk["source"],
```

```
Question: What is the purpose of randomization in a clinical trial?

======================================================================
Result 1
Source: doc2.txt
Chunk ID: doc2.txt_chunk_0
Score: 0.7555

Randomization and Control Groups

Randomization assigns participants to study groups using a chance-based method. Its purpose is to reduce selection bias and mak
e treatment groups more comparable.

A control group provides a comparison for the experimental treatment. A control may receive a placebo, standard treatment, or a
nother intervention.

Randomized controlled trials combine random assignment with a comparison group to improve the reliability of conclusions about
treatment effects.

======================================================================
Result 2
Source: doc1.txt
Chunk ID: doc1.txt_chunk_0
Score: 0.6433

Clinical Trial Phases

Phase I clinical trials primarily evaluate safety, dosage, and side effects. They usually involve a relatively small number of
participants.

Phase II trials evaluate whether a treatment shows evidence of effectiveness while continuing to monitor safety.

Phase III trials compare the treatment with existing treatments or standard care in larger groups of participants.

Phase IV studies occur after regulatory approval and continue to monitor long-term safety and effectivene

======================================================================
Result 3
Source: doc4.txt
```


4.5 Six Test Questions:

I created six test questions to evaluate different retrieval and generation scenarios. The questions were designed to cover a single-chunk answer, a two-chunk answer, similar

information across documents, an ambiguous question, a question whose answer is absent from the corpus, and an unrelated question.

Question	Purpose
Q1	Answer contained in one chunk
Q2	Answer requires two chunks
Q3	Similar information across documents
Q4	Ambiguous question
Q5	Answer not present in the documents
Q6	Unrelated question

Q1: What is the purpose of randomization in a clinical trial?
Q2: How do randomization and blinding help reduce bias in clinical research?
Q3: How is participant safety addressed across different stages and processes of a clinical trial?
Q4: What is the role of the study group?
Q5: What percentage of participants experience side effects during Phase I trials?
Q6: What is the capital of France?

Q5 and Q6 are expected to be refused by the context-engineered configuration because the required evidence is not available in the supplied document corpus.

4.6 Configuration A - No RAG:

Configuration A is the baseline. The question is sent directly to the qwen3:8b model without any retrieved document context. This configuration is used to compare the model's general response behavior against the grounded RAG approaches.

```
67  def configuration_a(llm, question):
68      prompt = f"""
69  Answer the following question clearly and concisely.
70
71  Question:
72  {question}
73  """
74
75      return llm.invoke(prompt).strip()
76
```

```
Blinding can reduce measurement and expectation bias when it is practical and appropriate for the study design.


CONFIGURATION A - NO RAG
--------------------------------------------------------------------------------
Approximately 30-50% of participants in Phase I trials experience side effects, though the exact percentage can vary depending
on the drug and study design. These side effects are typically mild to moderate and monitored closely to assess safety.

CONFIGURATION B - BASIC RAG
```

4.7 Configuration B - Basic RAG:

Configuration B retrieves the top 3 most similar chunks from the FAISS vector store and places the raw retrieved chunks directly into the prompt. The model then answers the question using the supplied retrieved context.

```python

def configuration_b(
    llm,
    question,
    retrievals
):
    context = format_retrievals(
        retrievals
    )

    prompt = f"""
Use the following retrieved context to answer
the question.

Context:
{context}

Question:
{question}

Answer:
"""

    return llm.invoke(prompt).strip()

```

```
CONFIGURATION B - BASIC RAG
--------------------------------------------------------------------------------
The provided context does not specify the percentage of participants who experience side effects during Phase I trials. It only
 states that Phase I trials "primarily evaluate safety, dosage, and side effects" and involve a "relatively small number of par
ticipants." No numerical data on side effect occurrence is included in the retrieved sources.

**Answer:** The context does not provide a specific percentage of participants experiencing side effects during Phase I trials.

CONFIGURATION C - CONTEXT ENGINEERED RAG
```

4.8 Configuration C - Context-Engineered RAG:

Configuration C improves the retrieved context before sending it to the model. I first remove duplicate chunks and then filter out lower-relevance chunks using their similarity scores. I used a minimum relevance score of 0.60 together with a relative threshold based on 90% of the highest retrieved score.

After filtering, the remaining chunks are ordered from highest to lowest similarity score and labeled with their source information. The model is instructed to answer only from this filtered context, cite the supporting source number, and refuse if there is not enough evidence in the documents.

This filtering helped reduce unnecessary context. For example, Q1 originally retrieved three chunks, but only the randomization chunk remained after filtering. For Q2, the system retained both the blinding and randomization chunks because both were relevant to the question. For the ambiguous Q4 question, none of the retrieved chunks passed the minimum relevance threshold, so the system refused instead of forcing an answer.

```python
def prepare_context_c(retrievals):
    # Step 1: remove duplicate chunks.
    cleaned_results = remove_duplicates(
        retrievals
    )

    # Step 2: remove lower-relevance chunks.
    cleaned_results, threshold = (
        remove_irrelevant_chunks(
            cleaned_results
        )
    )

    # Step 3: order surviving chunks by score.
    cleaned_results = sorted(
        cleaned_results,
        key=lambda item: item["score"],
        reverse=True,
    )

    return cleaned_results, threshold
```

```
CONFIGURATION C - CONTEXT-ENGINEERED RAG
--------------------------------------------------------------------------------
The purpose of randomization in a clinical trial is to reduce selection bias and make treatment groups more comparable [Source
1].

################################################################################
Q2: How do randomization and blinding help reduce bias in clinical research?
Question type: two chunks
################################################################################

TOP-3 RETRIEVED CHUNKS
--------------------------------------------------------------------------------
[Source 1]
Source: doc4.txt
Chunk ID: doc4.txt_chunk_0
Score: 0.8321
Blinding in Clinical Research

Blinding reduces the chance that expectations influence study results.

In a single-blind study, participants generally do not know which treatment group they are assigned to.

In a double-blind study, both participants and relevant study personnel are unaware of treatment assignments.

Blinding can reduce measurement and expectation bias when it is practical and appropriate for the study design.

[Source 2]
Source: doc2.txt
Chunk ID: doc2.txt_chunk_0
Score: 0.7537
Randomization and Control Groups
```

```
CONFIGURATION C - CONTEXT-ENGINEERED RAG
--------------------------------------------------------------------------------
I cannot answer this question from the provided documents

################################################################################
Q6: What is the capital of France?
Question type: unrelated
################################################################################

TOP-3 RETRIEVED CHUNKS
--------------------------------------------------------------------------------
[Source 1]
Source: doc3.txt
Chunk ID: doc3.txt_chunk_0
Score: 0.3098
Informed Consent

Informed consent is the process of providing potential research participants with information needed to decide whether to parti
cipate.

The consent process should explain the study purpose, procedures, possible risks, possible benefits, alternatives, and the part
icipant's right to withdraw.

Participation in a clinical trial should be voluntary. A participant may leave the study without being forced to continue.

[Source 2]
Source: doc5.txt
Chunk ID: doc5.txt_chunk_0
Score: 0.3072
Eligibility and Adverse Events

Eligibility criteria define who may participate in a clinical trial. Inclusion criteria describe characteristics participants m
ust have, while exclusion criteria identify conditions that prevent participation.

An adverse event is an unfavorable medical occurrence experienced by a participant during a study. An adverse event does not au
tomatically mean that the experimental treatment caused the event.

Researchers document and evaluate adverse events to support pa
```

```
================================================================================
RAG experiment complete.
Saved retrievals to: reports/hw04/raw/rag_retrievals.txt
Saved comparisons to: reports/hw04/raw/rag_comparison.txt
(.venv) (base) Triptis-MacBook-Air 🐼 ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(9h59m|git@hw4-work!)
```

```
Using the validity of conclusions about treatment effects.

CONFIGURATION C - CONTEXT-ENGINEERED RAG
--------------------------------------------------------------------------------
I cannot answer this question from the provided documents

################################################################################
Q5: What percentage of participants experience side effects during Phase I trials?
Question type: answer absent
################################################################################

TOP-3 RETRIEVED CHUNKS
--------------------------------------------------------------------------------
[Source 1]
Source: doc1.txt
Chunk ID: doc1.txt_chunk_0
Score: 0.7144
Clinical Trial Phases
```

```
CONFIGURATION C - FILTERED CONTEXT
--------------------------------------------------------------------------------
Relevance threshold: 0.7489
doc4.txt | doc4.txt_chunk_0 | score=0.8321
doc2.txt | doc2.txt_chunk_0 | score=0.7537
```

```
CONFIGURATION C - FILTERED CONTEXT
--------------------------------------------------------------------------------
Relevance threshold: 0.6000
No context retained / refusal required.
```

4.9 Top-K Sweep:

I evaluated the effect of retrieval context size using Q2, which asks how randomization and blinding reduce bias in clinical research. I tested k = 1, k = 3, and k = 5.

With k = 1, the retriever returned only the blinding document, so the context was incomplete because the randomization evidence was missing. With k = 3, the retriever returned both the blinding and randomization chunks needed to answer the question, along with one less relevant chunk. With k = 5, the two required chunks were still retrieved, but additional unrelated chunks about trial phases and informed consent were also included.

Therefore, k = 3 provided the best balance between sufficient evidence and limited irrelevant context. This experiment shows that increasing the amount of context does not always improve answer quality; larger k values can introduce unnecessary information into the model's context window.

k	***Retrieved Context***	***Result / Observation***
1	Blinding only	Incomplete; randomization evidence missing
3	Blinding + randomization + one less relevant chunk	Best balance; both required evidence sources retrieved
5	Blinding + randomization + three additional chunks	More context, but additional irrelevant information introduced

```
576  def run_k_sweep(
577      chunks,
578      index,
579      embeddings_model
580  ):
581      question = (
582          "How do randomization and blinding "
583          "help reduce bias in clinical research?"
584      )
585
586      output = []
587
588      for k in [1, 3, 5]:
589          results = retrieve(
590              question,
591              chunks,
592              index,
593              embeddings_model,
594              top_k=k
595          )
596
597          print()
598          print("=" * 80)
599          print(f"TOP-K SWEEP: k={k}")
600          print("=" * 80)
601
602          formatted = format_retrievals(results)
603
604          print(formatted)
605
606          output.append(
607              f"TOP-K SWEEP: k={k}\n"
```

```
================================================================================
TOP-K SWEEP: k=1
================================================================================
[Source 1]
Source: doc4.txt
Chunk ID: doc4.txt_chunk_0
Score: 0.8321
Blinding in Clinical Research

Blinding reduces the chance that expectations influence study results.

In a single-blind study, participants generally do not know which treatment group they are assigned to.

In a double-blind study, both participants and relevant study personnel are unaware of treatment assignments.

Blinding can reduce measurement and expectation bias when it is practical and appropriate for the study design.
```

```
================================================================================
TOP-K SWEEP: k=3
================================================================================
[Source 1]
Source: doc4.txt
Chunk ID: doc4.txt_chunk_0
Score: 0.8321
Blinding in Clinical Research

Blinding reduces the chance that expectations influence study results.

In a single-blind study, participants generally do not know which treatment group they are assigned to.

In a double-blind study, both participants and relevant study personnel are unaware of treatment assignments.

Blinding can reduce measurement and expectation bias when it is practical and appropriate for the study design.

[Source 2]
Source: doc2.txt
Chunk ID: doc2.txt_chunk_0
Score: 0.7537
Randomization and Control Groups

Randomization assigns participants to study groups using a chance-based method. Its purpose is to reduce selection bias
e treatment groups more comparable.

A control group provides a comparison for the experimental treatment. A control may receive a placebo, standard treatme
nother intervention.
```

```
================================================================================
TOP-K SWEEP: k=5
================================================================================
[Source 1]
Source: doc4.txt
Chunk ID: doc4.txt_chunk_0
Score: 0.8321
Blinding in Clinical Research

Blinding reduces the chance that expectations influence study results.

In a single-blind study, participants generally do not know which treatment group they are assigned to.

In a double-blind study, both participants and relevant study personnel are unaware of treatment assignments.

Blinding can reduce measurement and expectation bias when it is practical and appropriate for the study design.

[Source 2]
Source: doc2.txt
Chunk ID: doc2.txt_chunk_0
Score: 0.7537
Randomization and Control Groups

Randomization assigns participants to study groups using a chance-based method. Its purpose is to reduce selection bias
e treatment groups more comparable.

A control group provides a comparison for the experimental treatment. A control may receive a placebo, standard treatme
nother intervention.

Randomized controlled trials combine random assignment with a comparison group to improve the reliability of conclusion
treatment effects.

[Source 3]
Source: doc5.txt
```

```
Saved k-sweep results to: reports/hw04/raw/rag_k_sweep.txt
(.venv) (base) Triptis-MacBook-Air ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(10h56m|git@hw4-work!)
1447 ±
```

4.10 - Evaluation Table:

Question	Config	Correct Retrieval	Correct Answer	Grounded	Refused When Needed
Q1	A - No RAG	N/A	Yes	No	N/A
Q1	B - Basic RAG	Yes	Yes	Yes	N/A
Q1	C - Context RAG	Yes	Yes	Yes	N/A
Q2	A - No RAG	N/A	Yes	No	N/A
Q2	B - Basic RAG	Yes	Yes	Mostly Yes	N/A
Q2	C - Context RAG	Yes	Yes	Yes	N/A
Q3	A - No RAG	N/A	Partly	No	N/A
Q3	B - Basic RAG	Yes	Yes	Mostly Yes	N/A
Q3	C - Context RAG	Yes	Yes	Yes	N/A
Q4	A - No RAG	N/A	No	No	N/A
Q4	B - Basic RAG	Yes	Yes	Mostly Yes	N/A

Q4	C - Context RAG	No	Yes	Yes	N/A
Q5	A - No RAG	N/A	No	No	No
Q5	B - Basic RAG	Yes	Yes	Yes	Yes
Q5	C - Context RAG	Yes	Yes	Yes	Yes
Q6	A - No RAG	N/A	No	No	No
Q6	B - Basic RAG	No	No	No	No
Q6	C - Context RAG	No	Yes	Yes	Yes

Accuracy: The context-engineered RAG configuration produced the most reliable task-specific answers. It correctly answered supported questions and refused unsupported questions. The No-RAG baseline sometimes produced factually plausible answers, but it also introduced unsupported information and failed on ambiguous or out-of-corpus questions.

Faithfulness: Configuration C showed the strongest faithfulness because its answers were restricted to the retrieved context and included source citations. Basic RAG generally used the retrieved evidence but occasionally added explanatory details beyond the supplied text.

Format compliance: Configuration C followed the required source-citation format and used the exact refusal sentence for Q5 and Q6. Configuration A had no grounding format, while Configuration B did not consistently enforce the same strict refusal behavior.

Robustness: Configuration C handled supported, ambiguous, absent-answer, and unrelated questions more safely. For supported questions, it retained the strongest evidence and generated grounded answers. For Q4, the retrieved chunks did not meet the minimum relevance threshold, so the system refused rather than forcing an interpretation. It also correctly refused Q5 and Q6 when the required evidence was unavailable.

The evaluation results were also saved in *reports/hw04/raw/rag_evaluation.csv* for reproducibility and verification.

4.11 RAG Analysis:

The RAG experiment showed that the quality of retrieval and the way the context is given to the model both affect the final answer. For Q1, the correct randomization document was ranked first, so both Basic RAG and Context-Engineered RAG were able to give the correct

answer. The context-engineered version was better because it stayed concise and also cited the source.

Q2 required information from two different documents. The retriever correctly found the blinding document and the randomization document as the top two results. Because both pieces of information were available, the context-engineered version was able to explain that randomization helps reduce selection bias, while blinding helps reduce expectation and measurement bias.

Q3 showed how RAG can combine related information from different documents. The system retrieved information about trial phases as well as adverse events and eligibility. The No-RAG answer added extra information such as preclinical research and IRBs that was not present in the corpus. The context-engineered answer stayed closer to the retrieved evidence.

Q4 was useful for testing ambiguity. Without RAG, the model interpreted “study group” as an academic study group. Basic RAG used the clinical-trial context and interpreted it as a treatment/control group. However, in the context-engineered configuration, none of the retrieved chunks reached the minimum relevance threshold of 0.60. Instead of forcing an interpretation from weak evidence, the system refused to answer. This showed that filtering can make the system more cautious when the available context is ambiguous.

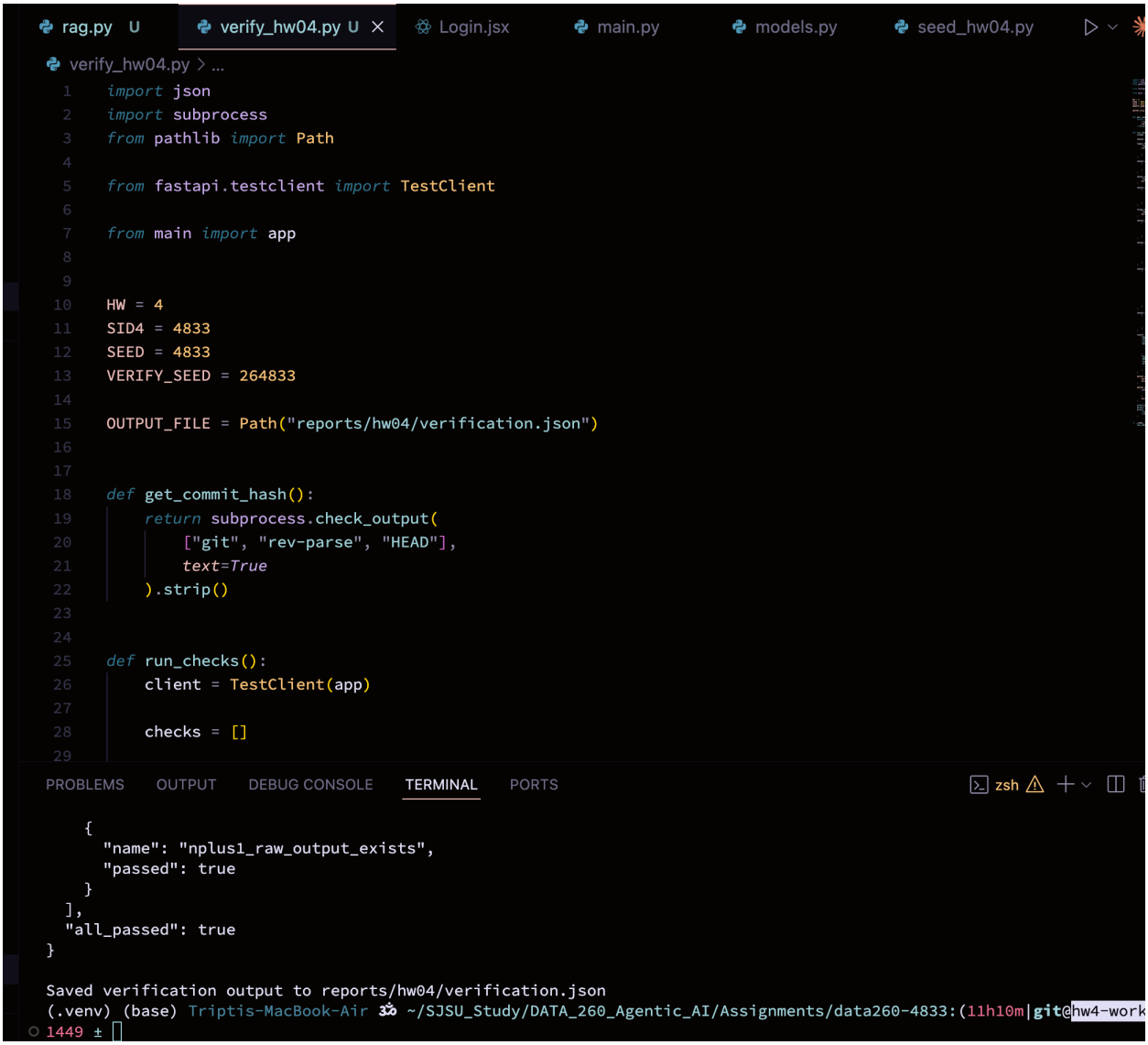
The relevance filtering also showed that context quality matters more than simply passing all retrieved chunks to the model. Q1 kept only its strongest chunk, while Q2 retained the two chunks needed to answer the question completely.

Q5 and Q6 also showed why grounding and refusal rules are important. For Q5, the No-RAG model produced an unsupported percentage even though the corpus did not contain that information. For Q6, No-RAG and Basic RAG used outside knowledge to answer “Paris.” In contrast, Configuration C refused both questions because the required evidence was unavailable. The top-k experiment also showed that more retrieved context is not always better: k=1 missed required evidence, k=3 provided the best balance, and k=5 added unnecessary chunks. Overall, the 500-character chunk size, source metadata, de-duplication, relevance filtering, ordering, and grounding instructions helped keep the final answers more focused and supported by the supplied documents.

Part 5

5.1 Verification / Smoke Test:

I created a HW4 smoke-test script to verify that the main application components still work together. The script checks successful authentication, the naive and fixed N+1 endpoints, consistency between their returned page sizes, and the presence of the required raw experiment files.

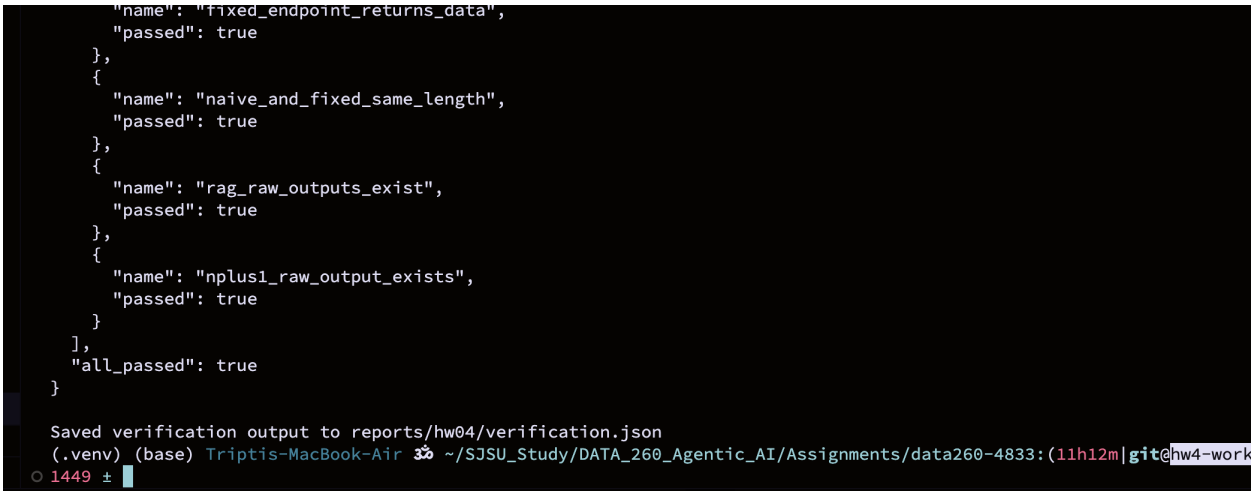


5.2 Verification Output:

The HW4 smoke test completed successfully. Authentication, both N+1 endpoints, output consistency, and the required raw experiment files were verified. All checks passed.

5.3 verification.json:

The verification script generated reports/hw04/verification.json, containing the homework number, SID4, current commit hash, model configuration, SEED, VERIFY_SEED, individual test results, and an overall all_passed: true status.



Part 6 : AI Use Disclosure

6.1 What I used AI for vs. what I did myself

I used ChatGPT and Claude mainly as learning and support tools during this homework. I used them to understand concepts such as React hooks and props, server-side sessions, the N+1 query problem, eager loading, FAISS retrieval, RAG configurations, and context engineering. I also used AI to better understand the assignment requirements, review my code, and get suggestions when I faced errors.

I completed the actual implementation and testing myself in my repository. I created and modified the project files, configured MySQL, ran the React and FastAPI applications, tested the APIs in Postman, created the database tables, generated the seed data, ran the N+1 benchmarks, completed the RAG experiments, collected screenshots, verified the outputs, and prepared the report. I also checked the AI suggestions against the actual results before using them in my final submission.

6.2 One AI-produced output that was wrong or unsuitable

While adding the fixed N+1 endpoint, the /fixed route was initially placed after the dynamic /{trial_id} route. Because of this, when I tested /api/hw4/trials/fixed, FastAPI treated the word fixed as the value of trial_id and returned a 422 validation error.

6.3 How I detected or verified the problem

I found this issue while testing the endpoint in Postman. The error response showed that FastAPI was trying to convert "fixed" into an integer for the trial_id path parameter. I then checked my router code and the order of the endpoint definitions, which helped me identify that the route order was causing the problem.

6.4 What I changed and why the corrected version works

I moved the /fixed endpoint above the /{trial_id} endpoint in routers/hw4_trials.py. After making this change, FastAPI correctly matched /fixed as its own static route before checking the dynamic trial ID route.

I tested the corrected endpoint again in Postman using page sizes 10, 50, and 200, and all requests returned 200 OK. I also included both the naive and fixed endpoints in the final verification script, and both checks passed successfully.